

Numerical Analysis Final Project

B12502027 李承軒

1.

(a) .

$$\text{兩直線: } \begin{cases} L_1: \frac{x-5}{3} = \frac{y+7}{-6} = \frac{z-1}{-2} \\ L_2: \frac{x-1}{3} = \frac{y}{2} = \frac{z+5}{2} \end{cases}$$

另 s 為直線 L_1 之參數 $\rightarrow L_1: \frac{x-5}{3} = \frac{y+7}{-6} = \frac{z-1}{-2} = s$, 因此位於直線 L_1

上之點 P 可表示為 $P(s) = (P_x, P_y, P_z) = (3s + 5, -6s - 7, -2s + 1)$

另 t 為直線 L_2 之參數 $\rightarrow L_2: \frac{x-1}{3} = \frac{y}{2} = \frac{z+5}{2} = t$, 因此位於直線 L_2 上

之點 Q 可表示為 $Q(t) = (Q_x, Q_y, Q_z) = (3t + 1, 2t, 2t - 5)$

這邊取距離或是距離平方為 cost function, 但由於後續會需利用取微分等於零之位置找到其最小距離點 (可想像此式並無最大值, 所以僅會有一個微分等於零的點, 此點即為距離最小處), 所以這邊使用無根號的距離平方為 cost function 以利後續微分方便。除此之外, 距離最小處與距離平方最小處應為相同位置, 所以使用距離平方為可行之方法。

$$\begin{aligned} \overline{PQ} &= \|P(s) - Q(t)\|_2 \\ &= \sqrt{(3s - 3t + 4)^2 + (-6s - 2t - 7)^2 + (-2s - 2t + 6)^2} \\ &= \sqrt{49s^2 + 17t^2 + 14st + 84s - 20t + 101} \end{aligned}$$

$$\begin{aligned} \text{另 cost function, } J(s, t) &= \overline{PQ}^2 = \|P(s) - Q(t)\|_2^2 \\ &= (3s - 3t + 4)^2 + (-6s - 2t - 7)^2 + (-2s - 2t + 6)^2 \\ &= 49s^2 + 17t^2 + 14st + 84s - 20t + 101 \end{aligned}$$

(b) .

觀察 cost function $J(s, t)$ 可發現其為好幾個平方相加, 類似於多變數之凹口向上碗狀曲面 (可排除微分等於零之位置為最大值或鞍點), 所以分別針對 s 以及 t 進行偏微分後取其兩者皆等於零時的參數代回 $P(s)$ 及 $Q(t)$ 位置式便可得其距離最小位置。

$$\begin{cases} \frac{\partial F(s,t)}{\partial s} = 98s + 14t + 84 = 0 \\ \frac{\partial F(s,t)}{\partial t} = 14s + 34t - 20 = 0 \end{cases} \rightarrow \begin{cases} 7s + t = -6 \\ 7s + 17t = 10 \end{cases}, \text{得} \begin{cases} s = -1 \\ t = 1 \end{cases},$$

$$\text{將} \begin{cases} s = -1 \\ t = 1 \end{cases} \text{代回原位置式} \begin{cases} P(s) = (3s + 5, -6s - 7, -2s + 1) \\ Q(t) = (3t + 1, 2t, 2t - 5) \end{cases} \rightarrow$$

$$\begin{cases} P(-1) = (2, -1, 3) \\ Q(1) = (4, 2, -3) \end{cases}, \text{兩點距離為最小值時 } cost = \|P(-1) - Q(1)\|_2^2 = (-2)^2 + (-3)^2 + 6^2 = 49,$$

$$\text{最短距離 } \overline{PQ} = \|P(-1) - Q(1)\|_2 = \sqrt{49} = 7.$$

(c) .

若假設點 P 與點 Q 重合，則必須滿足 $P(s) = Q(t)$ ，也就是三個座標

$$\text{分別相等} \begin{cases} 3s + 5 = 3t + 1 \\ -6s - 7 = 2t \\ -2s + 1 = 2t - 5 \end{cases} \rightarrow \begin{cases} 3s - 3t = -4 \\ -6s - 2t = 7 \\ -2s - 2t = -6 \end{cases}, \text{為方便觀察可寫成矩}$$

$$\text{陣形式} \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix} \begin{bmatrix} s \\ t \end{bmatrix} = \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix}, \text{並可表示為 } Au = b, A = \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix},$$

$$u = \begin{bmatrix} s \\ t \end{bmatrix}, b = \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix}.$$

若能找到一組參數 $u = \begin{bmatrix} s \\ t \end{bmatrix}$ 使得上方 $Au = b$ 成立，則可確定兩條直線會相交。然而若觀察此方程式可發現，題目中僅給予兩個未知數卻有三個互不相等之方程式，因此可判斷其為 overdetermined system，此情況下系統不一定會有精確解。而計算後可發現此題目 $Au = b$ 並未有精確解，即兩條直線並不會相交，點 P 與點 Q 重合無解。

(d) .

若一方程式由於等式的數量大於未知數而無法求得精確解時，可利用 left pseudo inverse 在最小平方誤差的情況下求得最理想的近似解：

由於 $Au = b$ 無法成立，我們可以令其殘值 *residue*, $r = Au - b$ ，此時

$|r|$ 數值越小，也就是 $\|Au - b\|^2$ 越小，則此參數 $u = \begin{bmatrix} s \\ t \end{bmatrix}$ 解便越貼近

我們想得到的近似解。沿用上面 cost function 的概念令 $J(u) =$

$(Au - b)^T(Au - b)$ ，由於此式為一平方和函數，所以最小值會出現在微

分等於零處，針對 $J(u)$ 進行微分並令其等於零， $\frac{\partial J(u)}{\partial u} = \frac{\partial}{\partial u}((Au - b)^T(Au - b)) = 2A^T(Au - b) = 0$ ，移項後可得 normal equation：
 $A^T Au = A^T b \rightarrow u = (A^T A)^{-1} A^T b$ ， $(A^T A)^{-1} A^T$ 即為 left pseudo inverse，
 而最小平方誤差解即為 $u = (A^T A)^{-1} A^T b$ 。

上面題目： $A = \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix}, u = \begin{bmatrix} s \\ t \end{bmatrix}, b = \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix}$

$$A^T = \begin{bmatrix} 3 & -6 & -2 \\ -3 & -2 & -2 \end{bmatrix}, A^T A = \begin{bmatrix} 49 & 7 \\ 7 & 17 \end{bmatrix}, A^T b = \begin{bmatrix} -42 \\ 10 \end{bmatrix}$$

$$\rightarrow \begin{bmatrix} 49 & 7 \\ 7 & 17 \end{bmatrix} \begin{bmatrix} s \\ t \end{bmatrix} = \begin{bmatrix} -42 \\ 10 \end{bmatrix} \rightarrow \begin{cases} s = -1 \\ t = 1 \end{cases}$$

此時其最小殘值 $r = Au - b = \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} - \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix} = \begin{bmatrix} -2 \\ -3 \\ 6 \end{bmatrix}$ ，距離為

$\|Au - b\|_2 = 7$ ，為其最短距離。

(e) .

在 Q1(a) 和 Q1(b) 中我們直接寫出參數式得出 $P - Q = (3s - 3t + 4, -6s - 2t - 7, -2s - 2t + 6)$ 並且 *cost function*, $J(s, t) =$

$$\|P(s) - Q(t)\|_2^2 = (3s - 3t + 4)^2 + (-6s - 2t - 7)^2 + (-2s - 2t + 6)^2,$$

直接寫出兩點距離的函式並針對距離平方求其最小值，目的是若能最小化此式子便能得到距離最近之解。而在 Q1(c) 和 Q1(d) 中，由上面的解釋可得知我們為了得到最理想的近似解，希望能讓殘值 $r = Au - b$ 的數值最小化，也就是說讓 $\|Au - b\|^2$ 最小化，若將參數皆代入式子

$$\|Au - b\|^2 = \left\| \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix} \begin{bmatrix} s \\ t \end{bmatrix} - \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix} \right\|^2 = (3s - 3t + 4)^2 + (-6s - 2t - 7)^2 + (-2s - 2t + 6)^2,$$

代入並比較兩者的式子後，我們便可發現兩者在最小化一模一樣之函式，故對此微分後所得之答案一定會相同。

2.

(a) .

$$y'' + 4y' + 4y = x, \quad y(0) = 0, \quad y(1) = 2$$

Homogeneous Solution : $y'' + 4y' + 4y = 0$, 其 Characteristic equation 為 $r^2 + 4r + 4 = 0 \rightarrow (r + 2)^2 = 0$, 可得 $r = -2$ 為重根解 , 因此其 Homogeneous Solution 為 $y_h = C_1 e^{-2x} + C_2 x e^{-2x}$ 。

Particular Solution : 由於等號右側為 x 一次項 , 所以我們可將 Particular Solution 假設為 $y_p = ax + b$, $y_p' = a$, $y_p'' = 0$, 將其代回原式 $\rightarrow 0 + 4a + 4ax + 4b = x$ 可得 $\begin{cases} a = \frac{1}{4} \\ b = -\frac{1}{4} \end{cases}$, 因此其 Particular Solution

$$\text{為 } y_p = \frac{1}{4}x - \frac{1}{4} \text{。}$$

General Solution : General Solution 為 Homogeneous Solution 和

Particular Solution 的線性組合 , $y = y_h + y_p = C_1 e^{-2x} + C_2 x e^{-2x} + \frac{1}{4}x - \frac{1}{4}$

$\frac{1}{4}$, 將題目條件 $y(0) = 0$, $y(1) = 2$ 代回 General Solution 可得

$$\begin{cases} y(0) = C_1 - \frac{1}{4} = 0 \\ y(1) = C_1 e^{-2} + C_2 e^{-2} + \frac{1}{4} - \frac{1}{4} = 2 \end{cases} \rightarrow \begin{cases} C_1 = \frac{1}{4} \\ C_2 = 2e^2 - \frac{1}{4} \end{cases} \text{ , 即 General}$$

$$\text{Solution 為 } y(x) = \frac{1}{4}e^{-2x} + (2e^2 - \frac{1}{4})xe^{-2x} + \frac{1}{4}x - \frac{1}{4} \text{。}$$

(b) .

(i).

我們首先會需要先定義一個 uniform grid (下面的 N 值為我們總共在此網格中取了幾個格點) , 且觀察題目條件可發現其皆為直接給定邊界上的函數值 (Dirichlet boundary condition) , $x_0 = 0$, $x_{N-1} = 1$ 分別為其兩端的邊界點 , 而所有點可表示為 $x_i = i\Delta x$, $i = 0, 1, 2, \dots, N-1$ with $\Delta x = \frac{1}{N-1}$, 由中央差分公式 (由一點兩側的資料 , 去近似這一點的導數) 可知

$$\text{針對一點 } x_i \text{ 其二階導數可近似為 } y''(x_i) \approx \frac{y_{i-1} - 2y_i + y_{i+1}}{\Delta x^2}, \quad i =$$

$$1, 2, \dots, N-2, \text{ 一階導數可近似為 } y'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2\Delta x}, \quad i = 1, 2, \dots, N-$$

2，將兩式代回原函式中 $\rightarrow \frac{y_{i-1}-2y_i+y_{i+1}}{\Delta x^2} + 4\frac{y_{i+1}-y_{i-1}}{2\Delta x} + 4y_i = x_i$ 化簡移

項後可得離散化結果 $y_{i-1} - 2y_i + y_{i+1} + 2\Delta x y_{i+1} - 2\Delta x y_{i-1} + 4\Delta x^2 y_i = \Delta x^2 x_i$

$\rightarrow \left(\frac{1}{\Delta x^2} - \frac{2}{\Delta x}\right)y_{i-1} + \left(4 - \frac{2}{\Delta x^2}\right)y_i + \left(\frac{1}{\Delta x^2} + \frac{2}{\Delta x}\right)y_{i+1} = x_i$ ，並可表示為

$ay_{i-1} + by_i + cy_{i+1} = x_i$, $a = \left(\frac{1}{\Delta x^2} - \frac{2}{\Delta x}\right)$, $b = \left(4 - \frac{2}{\Delta x^2}\right)$, $c =$

$\left(\frac{1}{\Delta x^2} + \frac{2}{\Delta x}\right)$ 。

最後將邊界值 $y_0 = 0$, $y_{N-1} = 2$ 代入後，可得到在矩陣形式中的第一列（即 $i = 1$ ）為 $by_1 + cy_2 = x_1 - ay_0 = x_1$ ，最後一列（即 $i = N - 2$ ）為 $ay_{N-3} + by_{N-2} = x_{N-2} - cy_{N-1} = x_{N-2} - 2c$ ，而在中間項（即 $i = 2, 3, 4, \dots, N - 1$ ）時每項為 $ay_{i-1} + by_i + cy_{i+1} = x_i$ ，將上方獲得之多條方程式寫成矩陣形式 $Ay = d$ 後，可得到其 linear equations 為

$$\begin{bmatrix} b & c & 0 & \cdots & 0 & 0 & 0 \\ a & b & c & \cdots & 0 & 0 & 0 \\ 0 & a & b & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & b & c & 0 \\ 0 & 0 & 0 & \cdots & a & b & c \\ 0 & 0 & 0 & \cdots & 0 & a & b \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-4} \\ y_{N-3} \\ y_{N-2} \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{N-4} \\ x_{N-3} \\ x_{N-2} - 2c \end{bmatrix}。$$

(ii) & (iii).

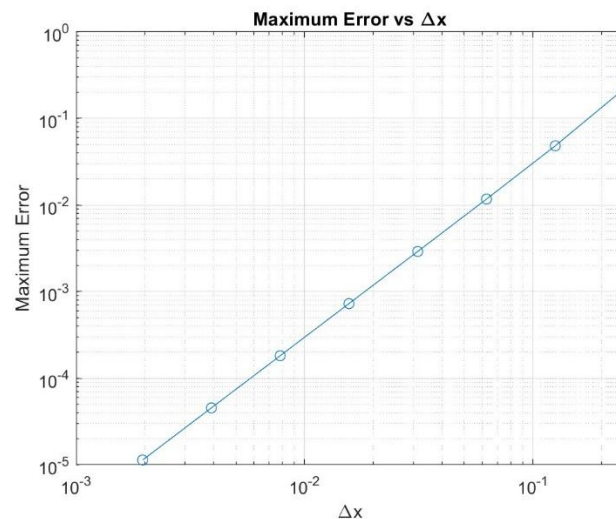
這邊使用 Gaussian Elimination 來解此線性方程組：Gaussian Elimination 的原理即為當我們得到一方程組 $Ay = d$ 後，我們利用列與列之間運算消除與簡化的方式將 A 矩陣簡化為相對簡單的上三角型式，再從最後一列的 y 開始計算並一步步回推出前面全部的解。而這邊在用 Gaussian Elimination 時有搭配上 partial pivoting，在進行運算消去前我們會先找目前這一欄裡絕對值最大的數字當作主元，若必要時我們會交換兩列，避免出現除以 0 或用太小的數字除的情況，可增加計算時的穩定性。而為什麼我在這邊要使用 Gaussian Elimination 我在後面會解釋。

這邊使用 MATLAB 程式協助我利用 Gaussian Elimination 計算矩陣，並將 Δx 設定為一變數並請程式輸出 Δx 、Maximum Error ($MAX(y_{Analysis} - y_{Numerical})$) 的表格以及關係圖協助我觀察在不同 Δx 下誤差的表現。

表一、不同 Δx 時 Maximum Error 對應表 (Gaussian Elimination)

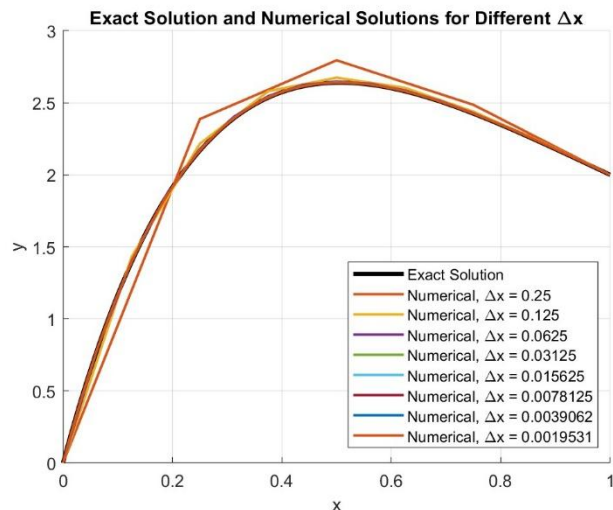
Δx	Max Error Value
0.25	0.21972
0.125	0.048279
0.0625	0.011721
0.03125	0.0029124
0.015625	0.00072788
0.0078125	0.00018189
0.0039062	4.5468e-05
0.0019531	1.1367e-05

這邊將 Δx 設定為 1/4、1/8、1/16、1/32、1/64、1/128、1/256、1/512，並分別列出在不同 Δx 下最大誤差的值，以利我觀察其誤差情況。



圖一、Maximum Error 與 Δx 關係圖 (Gaussian Elimination)

由上圖可發現，當網格切的越細，我們所得到之數值解與解析解之間的便會誤差越小，且由於兩者關係在 log-log 圖中成接近一條斜率為 2 的直線，可推測兩者為二次方的關係，亦即每次當 Δx 變為原本的 1/2 時，Maximum Error 會下降為原本的 1/4。這邊得出之結論與預期理論相同，由於這題我們在離散化時使用的是二階中央差分法，理論上使用中央差分法近似一階以及二階導數時誤差應為 $O(\Delta x^2)$ ，這也讓我們看到了這邊使用的有限差分法具有二階收斂的特性。



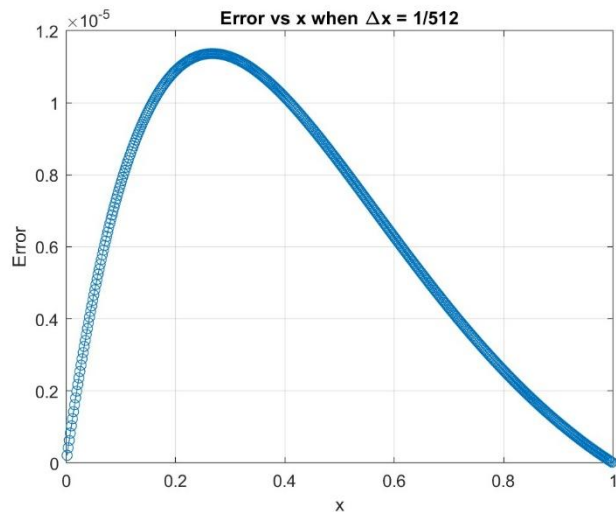
圖二、解析解以及不同 Δx 下數值解的比較圖（Gaussian Elimination）

表二、不同 Δx 時的誤差最大處以及誤差比例對應表（Gaussian Elimination）

Δx	誤差最大處	正確解 y_{exact}	數值解 $y_{numerical}$	誤差比例
1/4	$x = 0.25$	2.1671	2.3868	10.14%
1/8	$x = 0.25$	2.1671	2.2153	2.23%
1/16	$x = 0.25$	2.1671	2.1788	0.54%
1/32	$x = 0.28125$	2.2909	2.2938	0.13%
1/64	$x = 0.26563$	2.2320	2.2327	0.033%
1/128	$x = 0.26563$	2.2320	2.2322	0.0081%
1/256	$x = 0.26563$	2.2320	2.2320	0.0020%
1/512	$x = 0.26563$	2.2320	2.2320	0.0005%

這邊圖二我們將解析解以及不同 Δx 下的數值解進行疊圖比較，可發現若單純用肉眼觀察，由於其二階收斂的特性，當 $\Delta x = 1/4$ 時雖然誤差仍很大且偏移解析解非常多，但當 Δx 減半後誤差瞬間降為原本的 $1/4$ ，在圖中也可發現明顯貼近解析解許多，而當 Δx 再次減半後（即 $\Delta x = 1/16$ ）誤差已經降為原本的 $1/16$ ，在圖中已經幾乎和解析解幾乎重合，而在後續切得更細的網格中最大誤差已幾乎小於 10^{-3} ，相較於在最大誤差時的解析解數值少了接近三個數量級，可觀察到收斂速度非常快。

而觀察表二整理完的數據可發現，其最大誤差當 Δx 減半時，誤差大約下降 $1/4$ 左右，當網格切到 $\Delta x = 1/16$ 時誤差便已經低於 1% ，可發現其下降之快。



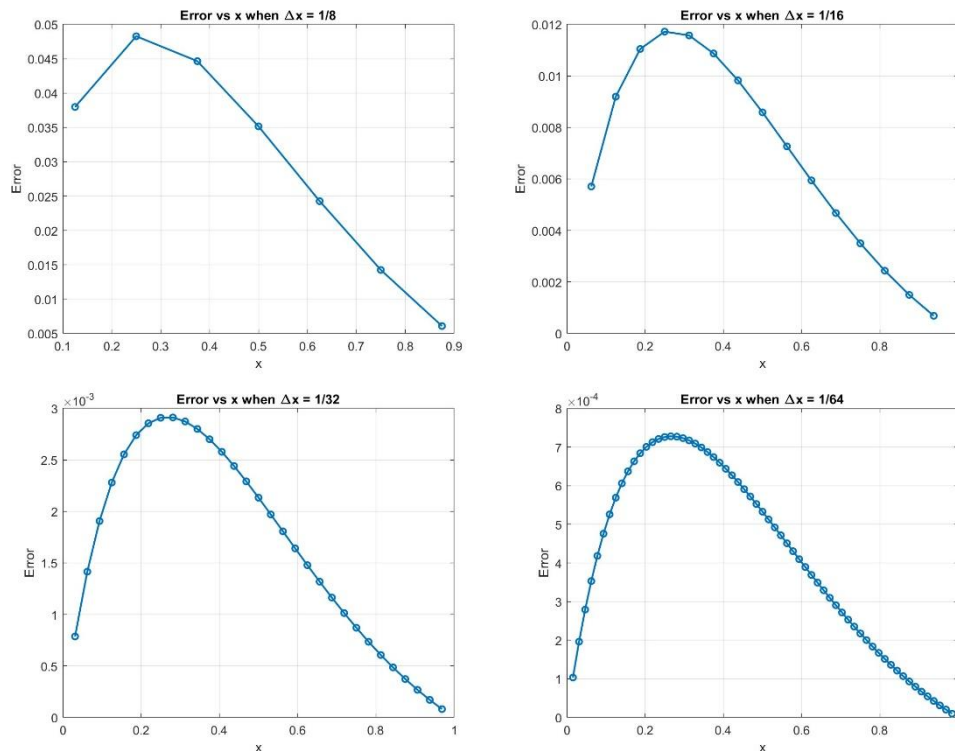
圖三、當 $\Delta x = 1/512$ 時的誤差絕對值分布（Gaussian Elimination）

我們在這邊特別將點切的最細以及誤差最小的 $\Delta x = 1/512$ 拿出來討論。可觀察到由於在兩側邊界，即 $x = 0$, $x = 1$ 時，數值並不是用逼近的，而是直接給定的，所以誤差會等於 0，而越往中間靠誤差會越大，最大值落在 $x = 0.265 \sim 0.269$ 的地方（雖然說此位置是誤差最大處，但也僅為 1.1367×10^{-5} ，還是比解析解 2.2397 少了約 10^5 倍），且整體呈現向 $x = 0$ 偏移的趨勢。推測其原因和整體 ODE 的結構有關，由於此方程式中存在了 $4y'$ 項，此一階導數項會讓整體方程式帶有類似偏移效果（由上方圖二也可看出此方程式明顯在靠近 $x = 0$ 時的變化幅度較 $x = 1$ 大），而方程式變化的幅度會進而影響到誤差的分布。

除此之外，也可觀察到當我們在寫出解析解 $y(x) = \frac{1}{4}e^{-2x} + (2e^2 -$

$\frac{1}{4})xe^{-2x} + \frac{1}{4}x - \frac{1}{4}$ 時，其中包含了 e^{-2x} 項，這個 e^{-2x} 在靠近 $x = 0$

時的變化比較明顯，往 $x = 1$ 移動的過程則會漸漸衰減，此行為讓整體函式在左側區域變化較為強烈，進而讓這區域的誤差容易產生並且放大。



圖四/圖五/圖六/圖七、不同 Δx 時的誤差絕對值分布（Gaussian Elimination）

後續針對較小的 Δx 輸出誤差絕對值分布，可發現即便 Δx 不如前面舉例的那麼大，誤差的分布圖形依舊長的差不多，為向 $x = 0$ 偏移的鐘形曲線，僅在縱軸誤差絕對值部分有較大的差異，此現象進而可推斷出此誤差分布為整體 ODE 的結構特性，和網格切的細緻程度較無關聯。

Gaussian Elimination 以及其他種計算線性方程組方法的比較：在教授的簡報中主要有提及四種方法，分別為 Gauss elimination、LU decomposition、Jacobi iteration、Gauss-Seidel iteration。從計算的過程可得知我們上面得到的結論——Maximum Error 和 Δx 呈二次方關係這件事情，和我們選用的方法沒有關聯，僅和我們離散化原方程式的過程有關，若我們在這邊使用了二階中央差分法，那不管我們這邊使用了教授教的哪種方法計算矩陣，我們得到的結論應該都會相同。而這邊之所以我們會採用 Gaussian Elimination，是因為這種方法能一次直觀的直接解出較精確的解，不用像 Jacobi iteration 和 Gauss-Seidel iteration 一樣經過多次迭代以逼近我們要的解，既麻煩又不夠精準，而 LU decomposition 則比較適合利用同組 A 去解多組的 d ，對於這題所需的運算沒有很大的幫助，所以我最後選擇最直觀且簡單易懂的 Gauss

elimination。然而 Gauss elimination 也有其缺點，對於一個 $n \times n$ 的矩陣，使用 Gauss elimination 的運算量大約是 $O(n^3)$ ，意思是若未知數變為 2 倍則運算量變為 8 倍，所以當網格切得非常細時，也就是 N 值非常大（例如 $N > 1000$ ）導致矩陣有很多項時，此方法的運算量便會變得非常大，但我們在這邊僅用到 $N = 513$ ，不至於會有運算量大到電腦卡頓的問題產生。

(c) .

(i).

由於此題給的條件分別落在 $x = 0$ 以及 $x = 1$ 兩邊界上，所以可認定為一 BVP，然而在許多數值方法中比較擅長解 IVP，所以這邊我們用 shooting method 將此 BVP 拆成兩個 IVPs，並利用規定的數值方法解此 IVPs。

Shooting method 概念：由於我們想強制將 BVP 轉為 IVP，然而我們在這邊僅有 $y(0) = 0$ 為已知， $y'(0) = 0$ 為未知。我們先假設一個初始

斜率 $y'(0) = \alpha$ 以滿足解 IVP 所需的兩個條件 $\begin{cases} y(0) = 0 \\ y'(0) = \alpha \end{cases}$ ，然後再從 $x = 0$ 開始一路向右求解至 $x = 1$ ，若一開始選的 α 太小，算到 $x = 1$ 時值可能會小於 2；若一開始選的 α 太大，則算到 $x = 1$ 時值可能會大於 2，我們再利用此性質不斷調整 α 使得當 $x = 1$ 能剛好滿足 $y(1) = 2$ 。

而在這題中，因為方程式是線性的，所以其實我們不用像前面說的一樣亂猜 α ，可以直接拆成兩個 IVPs：令 u 為 non-homogeneous IVP 的解； v 為 homogeneous IVP 的解，而由線性疊加特性可知其 IVP 的完整解為 $u + \theta v$ ， θ 為我們要找的調整係數，若將右邊界條件（即 $y(1) = 2$ ）代入，便可得到 θ 值。

原函式： $y'' + 4y' + 4y = x$, $y(0) = 0$, $y(1) = 2$

設 $w(x) = \begin{bmatrix} y(x) \\ y'(x) \end{bmatrix}$ ，可將方程式化簡為 $\frac{dw}{dx} = Aw + B =$

$\begin{bmatrix} 0 & 1 \\ -4 & -4 \end{bmatrix} \begin{bmatrix} y(x) \\ y'(x) \end{bmatrix} + \begin{bmatrix} 0 \\ x \end{bmatrix}$ ，由於此為一線性方程式，所以我們可以直接將

其解拆成兩個 IVPs，令 u_1 為 non-homogeneous IVP 的解， $u(x) =$

$\begin{bmatrix} u_1 \\ u_1' \end{bmatrix} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$, $\frac{du}{dx} = Au + B$, $u_1(0) = 0$, $u_2(0) = 0 \rightarrow u(0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ ；令 v_1

為 homogeneous IVP 的解， $v(x) = \begin{bmatrix} v_1 \\ v_1' \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$ ， $\frac{dv}{dx} = Av$ ， $v_1(0) =$

0 ， $v_2(0) = 1 \rightarrow v(0) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ ，由線性疊加特性可知其 IVP 的完整解為

$w = u + \theta v$ ，將右邊界條件（即 $y(1) = 2$ ）代入 $\rightarrow u_1(1) + \theta v_1(1) = 2$

推導可得 $\theta = \frac{2-u_1(1)}{v_1(1)}$ 即 $y(x) = u_1(x) + \frac{2-u_1(1)}{v_1(1)} v_1(x)$ 。

(ii).

這邊我們使用 Explicit Euler Method (one-step method) 來解此 IVPs，此方法簡單來說就是：下一點的值=現在的值+步長×現在的斜率，以向前

差分 $y'(t_n) = f(t_n, y_n) \approx \frac{y_{n+1}-y_n}{\Delta t}$ 為基本概念，推導後可得 $\rightarrow y_{n+1} =$

$y_n + \Delta t f(t_n, y_n)$ 。

將此方法應用在剛剛得到之兩個 IVPs（這邊我們將網格切成 N 等分， Δx 為步長，網格點 $x_n = n\Delta x$ ， $n = 0, 1, 2, \dots, N$ ）：

針對 non-homogeneous 解來說， $u_1' = u_2$ ， $u_2' = -4u_2 - 4u_1 + x$ ，將 Explicit Euler Method 代入可得

$$\begin{cases} u_{1,n+1} = u_{1,n} + \Delta x u_{2,n} \\ u_{2,n+1} = u_{2,n} + \Delta x (-4u_{2,n} - 4u_{1,n} + x) \end{cases}。$$

針對 homogeneous 解來說， $v_1' = v_2$ ， $v_2' = -4v_2 - 4v_1$ ，將 Explicit

$$\text{Euler Method 代入可得 } \begin{cases} v_{1,n+1} = v_{1,n} + \Delta x v_{2,n} \\ v_{2,n+1} = v_{2,n} + \Delta x (-4v_{2,n} - 4v_{1,n}) \end{cases}。$$

由上面推導可知 $\theta = \frac{2-u_1(1)}{v_1(1)}$ ， $u_1(1) = u_{1,N}$ ， $v_1(1) = v_{1,N}$ ，代入後可得

$$\theta_{\Delta x} = \frac{2-u_{1,N}}{v_{1,N}}$$

這邊 θ 會和 Δx 有關聯，當網格切的越細，從 $x = 0$ 一

步步算到 $x = 1$ 時得到的 $u_{1,N}$ 和 $v_{1,N}$ 誤差就越小， $\theta_{\Delta x}$ 也越準確。

(iii).

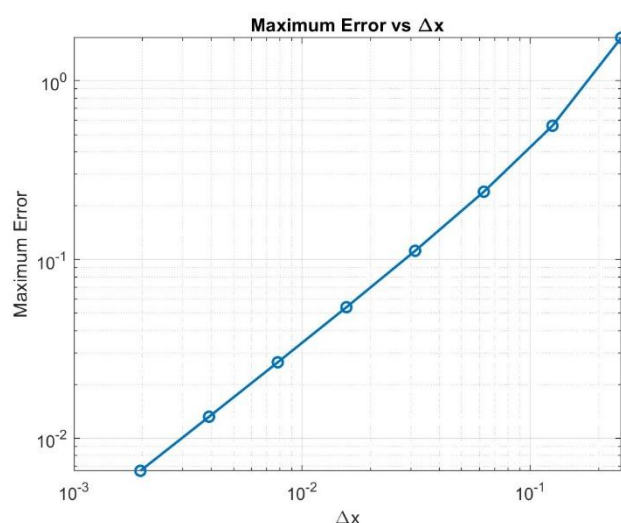
我在這邊請 MATLAB 程式利用 Explicit Euler Method 協助此 IVPs 的計算以及針對不同的 Δx 出圖以方便我做最後的誤差討論。

表三、不同 Δx 時 Maximum Error 對應表（Explicit Euler Method）

Δx	Max Error Value
0.25	1.7392
0.125	0.56023

0.0625	0.23937
0.03125	0.11182
0.015625	0.054072
0.0078125	0.026597
0.0039062	0.013192
0.0019531	0.0065691

這邊比照前面 Gaussian Elimination 的切點數將 Δx 一樣設定為 1/4、1/8、1/16、1/32、1/64、1/128、1/256、1/512，並輸出 Max Error 值。



圖八、Maximum Error 與 Δx 關係圖 (Explicit Euler Method)

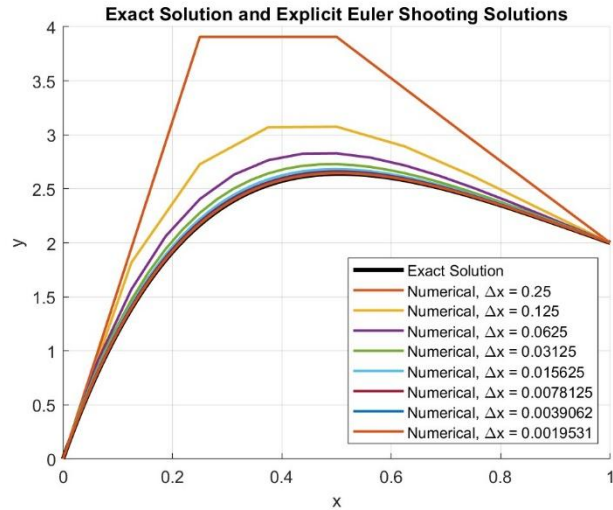
這邊可觀察到在使用 Explicit Euler Method 計算兩個 IVPs 並最後在合併成一個 BVP 的方法中，網格切的越細誤差越小，而其 Maximum Error 和 Δx 的關係一樣在 log-log 圖中會近似一條直線，然而此為一斜率為 1 的直線，可推測兩者為一次方的關係，亦即每次當 Δx 變為原本的 1/2 時，Maximum Error 也會下降為原本的 1/2。這邊得出之結論與預期理論相同，在 Explicit Euler Method 中，其 Truncation Error

(設從 x_n 到 x_{n+1}) 為 $y_{n+1} - (y_n + \Delta x f(x_n, y_n)) =$

$$\sum_{k=2}^{\infty} \frac{\Delta x^k}{k!} y^{(k)}(x_n) - (y_n + \Delta x y'(x_n)) = \sum_{k=2}^{\infty} \frac{\Delta x^k}{k!} y^{(k)}(x_n) \sim O(\Delta x^2), \text{ 若將}$$

$x = 0$ 到 $x = 1$ 的 Truncation Error 累計起來 $\rightarrow \frac{1}{\Delta x} \times O(\Delta x^2) \sim O(\Delta x)$ ，

可得理論上 Explicit Euler Method 的誤差會和 Δx 一次方成正比。這邊可發現我們使用 MATLAB 得出之結論與理論相符。



圖九、解析解以及不同 Δx 下數值解的比較圖 (Explicit Euler Method)

表四、不同 Δx 時的誤差最大處以及誤差比例對應表 (Explicit Euler Method)

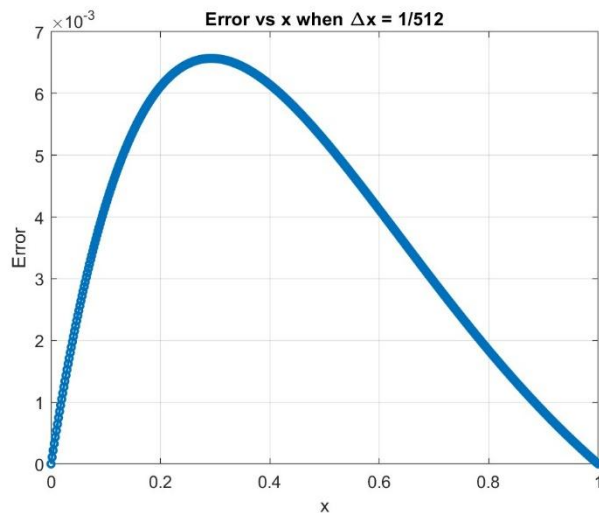
Δx	誤差最大處	正確解 y_{exact}	數值解 $y_{numerical}$	誤差比例
1/4	$x = 0.25$	2.1671	3.9062	80.25%
1/8	$x = 0.25$	2.1671	2.7273	25.85%
1/16	$x = 0.3125$	2.3920	2.6314	10.01%
1/32	$x = 0.28125$	2.2909	2.4027	4.88%
1/64	$x = 0.29688$	2.3442	2.3982	2.31%
1/128	$x = 0.28906$	2.3182	2.3448	1.15%
1/256	$x = 0.29297$	2.3314	2.3446	0.57%
1/512	$x = 0.29297$	2.3314	2.3379	0.28%

這邊我們將解析解以及不同 Δx 下的 Explicit Euler Method 數值解進行疊圖比較，可發現由上面的運算以及出圖可知此方法在誤差收斂上為一階收斂，所以當單純觀察圖九時，每當 Δx 減半其誤差收斂速度不會像 Gaussian Elimination 的二階收斂一樣明顯。

且觀察圖九可以看到一個特別的現象，就是上面 Gaussian Elimination 時誤差有正有負，但到了 Explicit Euler Method 可發現誤差僅有正偏差，我推測這邊的原因為由於在 Gaussian Elimination 時離散化的過程中使用的中央差分法，此方法同時參考了左右兩側格點資訊，因此誤差分布會較為對稱，在一些地方會大於解析解，一些地方則低於。然而 Explicit Euler Method 具有單方向性，概念是由左側邊界一路向右一步步運算，每一步都使用目前格點的斜率去預測下一個格點，因此誤差會沿

著運算方向累計並呈現固定方向的偏差，且由於此函式在前半段時斜率快速上升，到中間時才趨於平緩，所以在以舊斜率推估下一格時容易產生高估的情況。

而觀察表四可發現除了由於網格切的非常粗略（例如 $\Delta x = 1/4$ 和 $\Delta x = 1/8$ ）導致運算上非常不準確的幾個點之外（ $\Delta x = 1/4$ 時的誤差甚至飆高至 80.25%），其他誤差下降的速度和預期上差不多，大致上為當 Δx 降為原本的一半時，最大誤差也下降至原本的一半，而由於其受練速度僅為一階收斂，所以要當 $\Delta x = 1/256$ 時誤差才會下降至 1% 以下，收斂速度相對 Gaussian Elimination 慢上許多。而後續我也會針對所有的數值解法進行進一步的比較與分析。



圖十、當 $\Delta x = 1/512$ 時的誤差絕對值分布（Explicit Euler Method）這邊一樣輸出了網格最細的 $\Delta x = 1/512$ 時誤差絕對值對 x 分布，發現其誤差分布和 Gaussian Elimination 的分佈大致上相同，皆呈一偏向 $x = 0$ 的鐘形曲線，僅有在縱軸的部分數量級有明顯的差異而已。在之前 Q2b(ii)&(iii) 的敘述中已經可以得知，此誤差的分布現象與網格切的精度無關，而從這題中也可得到另一個新的結論便是此現象亦和計算的數值方法也無關，僅和方程式本身的結構有關聯。

(iv).

這邊使用的第二個 time-marching scheme 是 Crank–Nicolson Method (one-step method)。這個方法的概念和上面用的 Explicit Euler Method 相似，只是其概念變為：下一點的值=現在的值+步長×目前點與下一點斜率的平均值，其相較於 Explicit Euler Method 的優點是若一曲線在某段

區間斜率變化很快，Explicit Euler Method 在此時會有明顯誤差，但由於 Crank–Nicolson method 使用的是目前點跟下一點斜率的平均，所以其比較能反映這一小段區間內斜率的變化，得到的值相對也會較精準。由於 $y' = f(t, y)$ ，可將 y_n 以及 y_{n+1} 的關係表示為 $y_{n+1} = y_n +$

$\int_{t_n}^{t_n+\Delta t} f(t, y) dt$ ，若在積分處我們利用梯形法近似這個積分（即積分 $\approx \Delta t \times (\text{左端高度} + \text{右端高度})/2$ ），在這裡左端高度為 $f(t_n, y_n)$ 、右端高度為 $f(t_{n+1}, y_{n+1})$ ，代入可得 $y_{n+1} = y_n + \frac{\Delta t}{2}(f(t_n, y_n) + f(t_{n+1}, y_{n+1}))$ 。

將此方法應用在剛剛得到之兩個 IVPs（這邊我們將網格切成 N 等分， Δx 為步長，網格點 $x_n = n\Delta x$, $n = 0, 1, 2, \dots, N$ ）：

$$\text{回憶前面 } \frac{dw}{dx} = Aw + B(x) = \begin{bmatrix} 0 & 1 \\ -4 & -4 \end{bmatrix} \begin{bmatrix} y(x) \\ y'(x) \end{bmatrix} + \begin{bmatrix} 0 \\ x \end{bmatrix}$$

針對 non-homogeneous 解來說， $u(x) = \begin{bmatrix} u_1 \\ u_1' \end{bmatrix} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$, $u_1(0) =$

$$0, \quad u_2(0) = 0 \rightarrow u(0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \frac{du}{dx} = Au + B(x), \quad \text{將 Crank–Nicolson}$$

Method 代入可得 $u_{n+1} = u_n + \frac{\Delta x}{2}(Au_n + B(x_n) + Au_{n+1} + B(x_{n+1}))$ ，移

$$\text{向後化簡可得 } \rightarrow \left[I - \frac{\Delta x}{2}A\right]u_{n+1} = \left[I + \frac{\Delta x}{2}A\right]u_n + \frac{\Delta x}{2}(B(x_n) +$$

$$B(x_{n+1})), \quad I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}。$$

針對 homogeneous 解來說， $v(x) = \begin{bmatrix} v_1 \\ v_1' \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$, $v_1(0) = 0, \quad v_2(0) =$

$$1 \rightarrow v(0) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \frac{dv}{dx} = Av, \quad \text{將 Crank–Nicolson Method 代入可得}$$

$$v_{n+1} = v_n + \frac{\Delta x}{2}(Av_n + Av_{n+1}), \quad \text{移向後化簡可得 } \rightarrow \left[I - \frac{\Delta x}{2}A\right]v_{n+1} =$$

$$\left[I + \frac{\Delta x}{2}A\right]v_n, \quad I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}。$$

$$\text{令 } M_1 = I - \frac{\Delta x}{2}A, \quad M_2 = I + \frac{\Delta x}{2}A$$

$$\rightarrow \begin{cases} M_1 u_{n+1} = M_2 u_n + \frac{\Delta x}{2}(B(x_n) + B(x_{n+1})) \\ M_1 v_{n+1} = M_2 v_n \end{cases}, \quad \text{這邊的 } M_1 \text{ 以及 } M_2 \text{ 為兩}$$

個 2×2 的矩陣，這個方法為每一步皆計算一個線性系統來得到 u_{n+1}

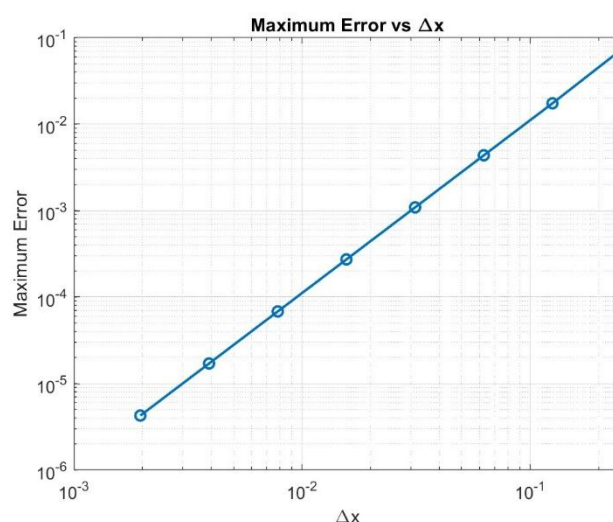
和 v_{n+1} ，算出 $\theta_{\Delta x} = \frac{2-u_{1,N}}{v_{1,N}}$ 後代回 $w = u + \theta v$ 便可得到 BVP 的解

$$\rightarrow y_n = u_{1,n} + \frac{2-u_{1,N}}{v_{1,N}} v_{1,n}, \quad n = 0, 1, 2, \dots, N。$$

我在這邊請 MATLAB 程式利用 Crank–Nicolson Method 協助此 IVPs 的計算以及針對不同的 Δx 出圖以方便我做最後的誤差討論。

表五、不同 Δx 時 Maximum Error 對應表 (Crank–Nicolson Method)

Δx	Max Error Value
0.25	0.072746
0.125	0.017444
0.0625	0.004365
0.03125	0.0010903
0.015625	0.00027264
0.0078125	6.815e-05
0.0039062	1.7039e-05
0.0019531	4.2596e-06



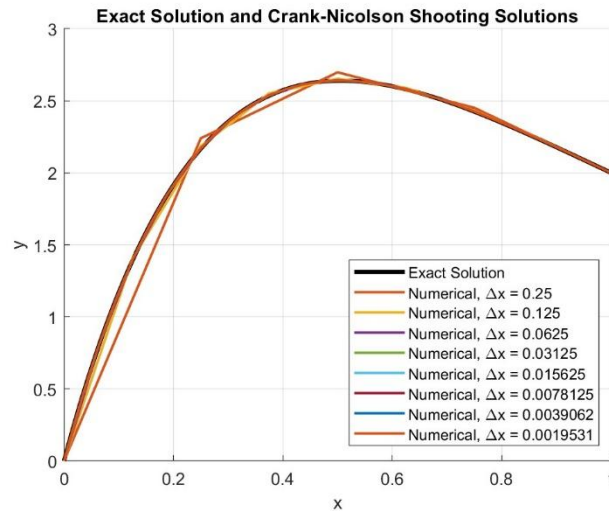
圖十一、Maximum Error 與 Δx 關係圖 (Crank–Nicolson Method)

趨勢和前面都一樣，網格越密最大誤差便會越小，而觀察上圖可發現在 Crank–Nicolson Method 中，其最大誤差和 Δx 在 log-log 圖中是近似於一斜率為 2 的直線，可推測兩者關係為二次方的關係，即 Δx 變為原本的 1/2 時誤差會變為原本的 1/4。從理論上來說，Crank–Nicolson Method 的 Truncation Error (設從 x_n 到 x_{n+1}) 為 $y_{n+1} - y_n -$

$$\frac{\Delta x}{2} (f(x_n, y_n) + f(x_{n+1}, y_{n+1})) = \sum_{k=3}^{\infty} \frac{\Delta x^k}{k!} y^{(k)}(x_n) -$$

$$\sum_{k=2}^{\infty} \frac{\Delta x^{k+1}}{2 \times k!} y^{(k+1)}(x_n) = \sum_{k=3}^{\infty} \left(\frac{1}{k} - \frac{1}{2}\right) \frac{\Delta x^k}{(k-1)!} y^{(k)}(x_n) \sim O(\Delta x^3), \text{ 若將 } x = 0$$

到 $x = 1$ 的 Truncation Error 累計起來 $\rightarrow \frac{1}{\Delta x} \times O(\Delta x^3) \sim O(\Delta x^2)$ ，由理論可得知 Crank–Nicolson Method 的誤差會合 Δx^2 相關，與當時我們計算並出圖後得到的結果呈一致。



圖十二、解析解以及不同 Δx 下數值解的比較圖 (Crank–Nicolson Method)

表六、不同 Δx 時的誤差最大處以及誤差比例對應表 (Crank–Nicolson Method)

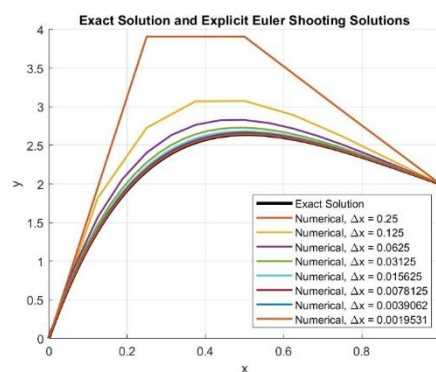
Δx	誤差最大處	正確解 y_{exact}	數值解 $y_{numerical}$	誤差比例
1/4	$x = 0.25$	2.1671	2.2398	3.36%
1/8	$x = 0.25$	2.1671	2.1845	0.80%
1/16	$x = 0.3125$	2.3920	2.3964	0.18%
1/32	$x = 0.28125$	2.2909	2.2920	0.048%
1/64	$x = 0.29688$	2.3442	2.3444	0.012%
1/128	$x = 0.28906$	2.3182	2.3183	0.0029%
1/256	$x = 0.29297$	2.3314	2.3314	0.0007%
1/512	$x = 0.29297$	2.3314	2.3314	0.0002%

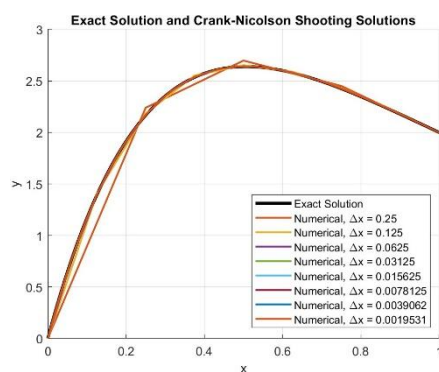
觀察 Crank–Nicolson Method 的誤差表現，可發現其在計算的精確度上相較於 Explicit Euler Method 好上非常多，在網格精度最差 ($\Delta x = 1/4$) 的情況下誤差仍只有 3.36%，且由於其二階收斂的特性，在圖十二中可觀察出其收斂的速度非常快，當 $\Delta x = 1/16$ 時其算出的值便與

除此之外，觀察圖十二可發現其誤差和 Gaussian Elimination 一樣呈有正有負，推測其原因為與 Explicit Euler Method（僅用單邊預測的從左向右推）不同的是 Explicit Euler Method 會將此計算點的左右兩端皆納入考量，即平均斜率 = (左端點斜率 + 右端點斜率) / 2，所以其誤差並不會呈現固定往上或固定往下的趨勢，整體誤差便會有正有負。

A log-log plot showing the Maximum Error (Y-axis, ranging from 10^{-6} to 10^1) versus Δx (X-axis, ranging from 10^{-1} to 10^{-5}). The plot compares the performance of three numerical methods: Gaussian Elimination (blue line with circles), Explicit Euler Method (orange line with circles), and Crank-Nicolson Method (green line with circles). The Explicit Euler Method shows the highest error, followed by Gaussian Elimination, and then Crank-Nicolson Method, which shows the lowest error across the range of Δx values.

Δx	Gaussian Elimination	Explicit Euler Method	Crank-Nicolson Method
10^{-1}	1.1×10^{-5}	7.0×10^{-2}	4.5×10^{-6}
10^{-2}	5.0×10^{-5}	1.5×10^{-1}	1.8×10^{-5}
10^{-3}	2.5×10^{-4}	3.0×10^{-1}	7.0×10^{-5}
10^{-4}	1.2×10^{-3}	6.0×10^{-1}	2.8×10^{-4}
10^{-5}	6.0×10^{-3}	1.2×10^0	1.0×10^{-3}





圖十四/圖十五/圖十六、解析解以及不同 Δx 下數值解的比較圖（三種方法比較）

表七、不同 Δx 時的誤差最大處的誤差比例對應表（三種方法比較）

Δx	Gaussian Elimination 誤差比例	Explicit Euler Method 誤差比例	Crank–Nicolson Method 誤差比例
1/4	10.14%	80.25%	3.36%
1/8	2.23%	25.85%	0.80%
1/16	0.54%	10.01%	0.18%
1/32	0.13%	4.88%	0.048%
1/64	0.033%	2.31%	0.012%
1/128	0.0081%	1.15%	0.0029%
1/256	0.0020%	0.57%	0.0007%
1/512	0.0005%	0.28%	0.0002%

圖十三將上述三種方法的 Maximum Error 與 Δx 關係圖進行疊圖輸出並觀察其差異，可發現當網格切的很粗糙時，Explicit Euler Method 的誤差相較於另外兩種方法顯得非常大，在誤差最大值時誤差比例高達 80.25%，相較於 Gaussian Elimination 的 10.14% 和 Crank–Nicolson Method 的 3.36% 差了至少 8 倍，由於 Explicit Euler Method 目前點的斜率去推估下一點，所以當網格切的非常粗糙時，誤差會特別明顯，而另外兩種方法會將多點納入考量，比較容易能對函式的變化做出反應，誤差便相對較小，而其中又以 Crank–Nicolson Method 的 3.36% 最小。而觀察斜率可發現，由前面的推倒可得知 Gaussian Elimination 和 Crank–Nicolson Method 的誤差皆呈現二次收斂 ($O(\Delta x^2)$)，Explicit Euler Method 則是呈現一次收斂 ($O(\Delta x)$)，所以在將 Δx 切更細的過程中，Gaussian Elimination 和 Crank–Nicolson Method 的誤差下降速度會較 Explicit Euler Method 快上許多，當網格切到 $\Delta x=1/512$ 時 Gaussian

Elimination 和 Crank–Nicolson Method 的誤差比例已經降至 10^{-4} 量級，Explicit Euler Method 甚至還停留在 10^{-1} 量級，這結論也可藉由比較圖十四/圖十五/圖十六和觀察表七得知。

3.

(1).

每個元素的 element stiffness matrix K^e 描述了這個元素的節點位移和節點力之間的關係，可表示為 $K^e u^i = F^e$ ，且由於我們在這邊關注的都是二維平面問題，每個節點都有兩個方向的自由度，而一個 four-node quadrilateral element 有四個節點，所以一個元素總共有 $4 \times 2 = 8$ 個自由度， K^e 通常會是一個 8×8 的矩陣，最後我們再將每個元素的 K^e 合併成整體結構的剛性矩陣 K 後便可得 $KU = F$ ，當節點數越多時此 K 矩陣即會越大。

在了解 K 矩陣的概念之後我們可以觀察其性質並統整為以下：

- **Sparse matrix（稀疏矩陣）和 Banded matrix（帶狀矩陣）：**

組裝後之 K 矩陣為一稀疏矩陣以及帶狀矩陣，即矩陣裡面大部分元素都是 0 且非 0 項大多集中在對角線附近。原因是有限元素法中每個元素都只會連到自己附近的幾個節點，例如一個 four-node quadrilateral element 只會影響到它連接的四個節點，而這個元素的 K^e 便只影響這四個點的自由度，再加上編號時節點編號通常會按照空間位置排列，所以當把所有 K^e 合併起來後，非 0 項會出現在相鄰或是屬於同一個元素的節點所對應的自由度位置上，並且由於節點編號通常會依照空間位置排列，所以這些自由度的編號通常相近，讓非 0 項集中在主對角線附近，而對角線附近以外的區域便是 0。例如假設有一個 100×100 的 K 矩陣， $K_{60,40}$ 代表第 60 個自由度與第 40 個自由度之間的剛性關聯，如果這兩個自由度所對應的節點在空間上距離比較遠且不屬於同一個元素的話，它們之間通常沒有直接的剛性關聯，因此此位置的值通常是 0。

- **Symmetric matrix（對稱矩陣）：**

組裝後之 K 矩陣為一對稱矩陣，即 $K_{i,j} = K_{j,i}$ 。觀察公式 $K^e =$

$\int_{D_k} B^T D B dV$ 可知，由於一般彈性材料的材料矩陣 D 是對稱的，所

以由此公式推導出的 K^e 也會是對稱的，在將 K^e 合併成之 K 矩

陣便會是對稱矩陣，意思是自由度 i 的位移會對自由度 j 的力產生影響，反過來自由度 j 的位移也會對自由度 i 的力產生影響。

- 在沒有邊界條件之前 K 矩陣為 **Positive semi-Definite**（半正定），但加上邊界條件之後 K 矩陣變為 **Positive Definite**（正定）：

結構力學中應變能 $= \frac{1}{2} d^T K d$ ， d 為所有節點位移組成的向量，而

此應變能只會是正的 $\rightarrow \frac{1}{2} d^T K d \geq 0$ ，並不會出現變形後能量 < 0

（不做功就能自己產出能量的概念）的情況，然而當我們在加入邊界條件之前， d 可能代表的是鋼體運動（例如整個結構一起位移），此時整體結構就沒有形變，應變能便會等於 0，因此剛性矩陣 K 具有 **Positive semi-Definite** 的性質且此時系統並無唯一解。然而我們在加入邊界條件（例如固定懸臂樑左端點 $u_1 = 0$ ， $u_2 = 0$ ）之後，我們便可以排除鋼體運動的可能，結構無法做到一起位移、旋轉，所以只要 d 不是零向量， $\frac{1}{2} d^T K d$ 都會大於 0，剛性矩陣 K 便會具有 **Positive Definite** 的性質，系統此時便會有唯一解。

(2) .

由 (1) 可得知 K 矩陣為一 **Sparse**、**Banded**、**Symmetric** 且當加入邊界條件之後是 **Positive Definite** 的矩陣。而這邊我們之所以不使用一般的 **Gauss elimination** 來解此線性系統的原因為記憶體以及計算時間的浪費，由上方 Q2(b) 可知針對於一個 $n \times n$ 的矩陣，使用 **Gauss elimination** 的運算量大約是 $O(n^3)$ ，所以像這種由非常多元素組成一個整體再求解其整體線性系統的方式，節點數越多時會讓矩陣 K 過於龐大，讓運算量以及運算時間非常大，除此之外， K 矩陣為一 **Sparse matrix**，存在許多的 0 項，所以在使用 **Gauss elimination** 時會需要用到多次的 **partial pivoting**，避免出現除以 0 的情況，影響計算的穩定性。而在記憶體的部分，**Sparse matrix** 中大部分的項皆為 0，若我們花上大量的記憶體去儲存這些 0 會讓整體空間使用上很沒效率。

- 如果問題規模不大時可以用 **sparse Cholesky decomposition**：

若將邊界條件加入之後， K 矩陣便會變成一個 **Symmetric Positive Definite matrix**，解這種矩陣直接的方法就是教授簡報裡的 **Cholesky decomposition**，此方法先將 K 矩陣分解為 $K = LL^T$ ，線性系統便

會變成 $\rightarrow LL^T U = F$ ，此時我們假設 $L^T U = y$ ，先利用 Forward Substitution 計算 $Ly = F$ ，再利用 Backward Substitution 計算 $L^T U = y$ ，這邊由於 L 為一下三角矩陣，所以兩個計算都不複雜，而因為 K 矩陣為一 Sparse matrix，如果我們用單純的 Cholesky decomposition 會將所有的數值存皆下來（包含 0 和非 0 項），空間使用上很沒效率，所以我們這邊採用 sparse Cholesky decomposition，僅存取非 0 項，減少記憶體浪費。

- **如果問題規模很大時可以改用 Conjugate Gradient method：**

然而若矩陣進一步擴大之後，sparse Cholesky decomposition 的缺點便會變得很明顯。在進行計算的時候，雖然 K 矩陣有很多位置是 0，但其分解完之後的 L 所對應的位置不一定是 0，造成 L 矩陣可能會比原本的 K 矩陣更加的密集，此時當問題規模變得很大（即 K 矩陣變得很大）的話記憶體和計算量都會變得過於龐大。

而若其問題規模很大，這邊可以改用 Conjugate Gradient method，這方法適合用在大規模的稀疏對稱正定線性方程組，而其運算的邏輯是先猜一組 U ，然後計算誤差 *residue*, $r = F - KU$ 並沿著適合的方向修正 U ，重複直到誤差在可接受範圍，為一迭代法。這邊之所以適合這個方法是因為每次 Conjugate Gradient method 計算時不需要完整的分解矩陣，只需要反覆做矩陣向量的乘法就好，而由於 K 是一個 Sparse matrix，所以進行這種乘法會非常快。

(3) .

- **Displacement Error：**

我們這邊使用的是 four-node quadrilateral element，且每個元素的位移可由其四個節點加權組成（即為 shape function， N_i ），而這四個

節點的 shape function 可表示為

$$\begin{cases} N_1(\xi_1, \xi_2) = \frac{1}{4}(1-\xi_1)(1-\xi_2) \\ N_2(\xi_1, \xi_2) = \frac{1}{4}(1+\xi_1)(1-\xi_2) \\ N_3(\xi_1, \xi_2) = \frac{1}{4}(1+\xi_1)(1+\xi_2) \\ N_4(\xi_1, \xi_2) = \frac{1}{4}(1-\xi_1)(1+\xi_2) \end{cases}, \quad -1 \leq \xi_1 \leq 1, \quad -1 \leq \xi_2 \leq 1$$

1, $-1 \leq \xi_2 \leq 1$ ，若 N_i 越大便可知節點 i 對於某個元素位移影響越大，且可發現這些函式都是 bilinear shape function，即在 ξ_1 方向上和 ξ_2 皆為線性變化，此重要性質可協助我們後面誤差的預測。除此之外，節點 1 在此自然座標中的位置為 $(\xi_1, \xi_2) = (-1, -1)$ ，若將此座標代入後可發現在節點 1 的位置其 $N_1 = 1$ ，

而其他 N_2, N_3, N_4 皆等於 0，可知在節點 1 的位置其位移完全由節點 1 控制，節點 2、3、4 也同理。

假設每個元素的尺寸為 h ，如果我們這邊觀察在一維狀態下一小段元素真實位移 $u(x)$ 的泰勒展開式 $u(x) \approx u(x_0) + u'(x_0)(x - x_0) + \frac{1}{2}u''(x_0)(x - x_0)^2 + H.O.T$ ，而由上面的解釋可得知，我們這邊元素

位移的近似利用有限元素法也可表示為 $u_h(\xi_1, \xi_2) = \sum N_i(\xi_1, \xi_2)u_i$ ，即為數值位移 = shape function $\times$ 節點位移，但由於我們的 shape function 為 bilinear 的，在個別方向上都僅為一次函數，所以這邊

如果我們用數值方法的話會無法表示 $\frac{1}{2}u''(x_0)(x - x_0)^2 + H.O.T$ 以

後的項次，這便是誤差的來源，我們在這邊便可得出其

Displacement Error $\|u - u_h\| \propto h^2$ ，即 $O(h^2)$ 。

● Strain/Stress Error：

由題目中可知應變為 B 矩陣乘上節點位移，而 B 矩陣裡面是

$$\text{shape function 對空間的導數，可表示為} \rightarrow \begin{bmatrix} \epsilon_1(x_1, x_2) \\ \epsilon_2(x_1, x_2) \\ \gamma_{12}(x_1, x_2) \end{bmatrix} = \begin{bmatrix} \frac{\partial N_1}{\partial x_1} & 0 & \frac{\partial N_2}{\partial x_1} & 0 & \frac{\partial N_3}{\partial x_1} & 0 & \frac{\partial N_4}{\partial x_1} & 0 \\ 0 & \frac{\partial N_1}{\partial x_2} & 0 & \frac{\partial N_2}{\partial x_2} & 0 & \frac{\partial N_3}{\partial x_2} & 0 & \frac{\partial N_4}{\partial x_2} \\ \frac{\partial N_1}{\partial x_2} & \frac{\partial N_1}{\partial x_1} & \frac{\partial N_2}{\partial x_2} & \frac{\partial N_2}{\partial x_1} & \frac{\partial N_3}{\partial x_2} & \frac{\partial N_3}{\partial x_1} & \frac{\partial N_4}{\partial x_2} & \frac{\partial N_4}{\partial x_1} \end{bmatrix} \begin{bmatrix} u_1^1 \\ u_2^1 \\ u_1^2 \\ u_2^2 \\ u_1^3 \\ u_2^3 \\ u_1^4 \\ u_2^4 \end{bmatrix}, \text{而再由上面推導可}$$

知其位移誤差為 $O(h^2)$ ，我們將此對空間進行一次微分之後可得 $\rightarrow$

$$\frac{\partial(u - u_h)}{\partial x} \propto h, \text{誤差階數會降一階，可知 strain error} = O(h)。$$

應力的部分由題目中可知 $\sigma = D\epsilon$ ，這邊的 D 為材料矩陣，並不會收斂階數，所以應力的誤差會和應變誤差同階，即 stress error = $O(h)$ 。

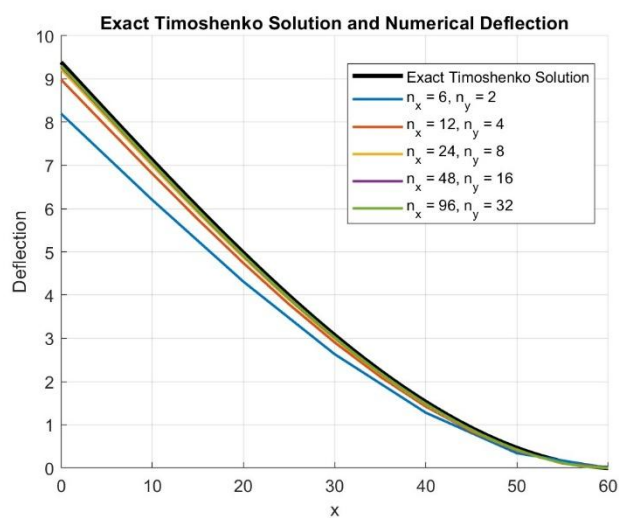
(4) .

這邊我們利用 MATLAB 協助進行題目模擬、誤差的計算以及分析，網

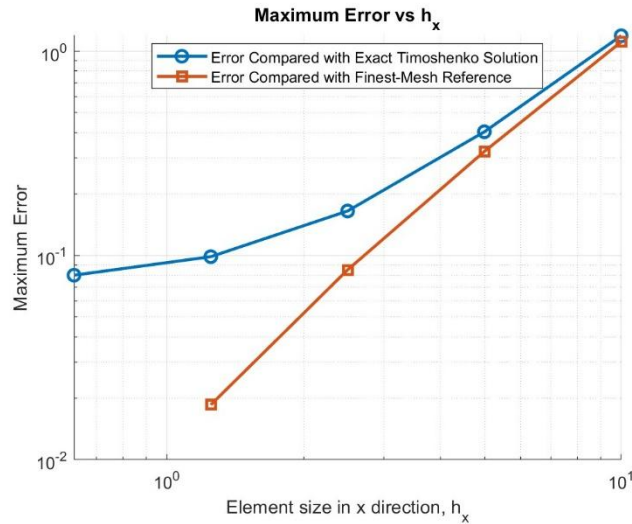
格採用 $(N_x, N_y) = (6, 2) \cdot (12, 4) \cdot (24, 8) \cdot (48, 16) \cdot (96, 32)$ ，再細的話電腦就有點跑不動了，且參考題目設定的系統邊長比例關係，此 $N_x:N_y = 3:1$ 的網格數量比例會讓每個元素皆為邊長 h 的正方形。這邊我們輸出了網格數、元素邊長 h 、以及 Numerical Tip Displacement（數值解位移）、Exact Tip Displacement（正解位移），而在誤差的部份，我們除了輸出 FEM 解和正解的誤差之外，額外以最細網格 (96×32) 為參考，輸出了每個粗網格 FEM 解和最細網格 FEM 解的誤差，並觀察分析這兩種誤差的表現。

表八、不同網格下的位移以及誤差數值

N_x	N_y	h (mm)	數值解最大位移 (mm)	正解最大位移 (mm)	正解最大誤差 (mm)	最細網格參考誤差 (mm)
6	2	10	8.1981	9.3888	1.1907	1.1106
12	4	5	8.9855	9.3888	0.4033	0.3232
24	8	2.5	9.2237	9.3888	0.1651	0.08506
48	16	1.25	9.2901	9.3888	0.09865	0.01858
96	32	0.625	9.3087	9.3888	0.08008	<i>NaN</i>



圖十七、不同網格下位移以及位置關係圖



圖十八、最大誤差和元素邊長 h 關係圖

表八將不同網格下的誤差數值計算出來，可發現當網格越細時，其數值解跟正解之間的誤差便會越小，且由圖十七可看出當網格密度增加時，其位移和位置的關係曲線會越貼近正解的曲線，當網格最粗糙時（即 6×2 ），正解最大誤差比例高達 $\frac{1.1907}{9.3888} = 12.682\%$ ，而當網格切的最密時

（即 96×32 ），正解最大誤差比例僅為 $\frac{0.08008}{9.3888} = 0.853\%$ ，兩者相差了

兩個數量級，且當網格切至 96×32 時，圖十七中的曲線已經和正解曲線大致上重合了。

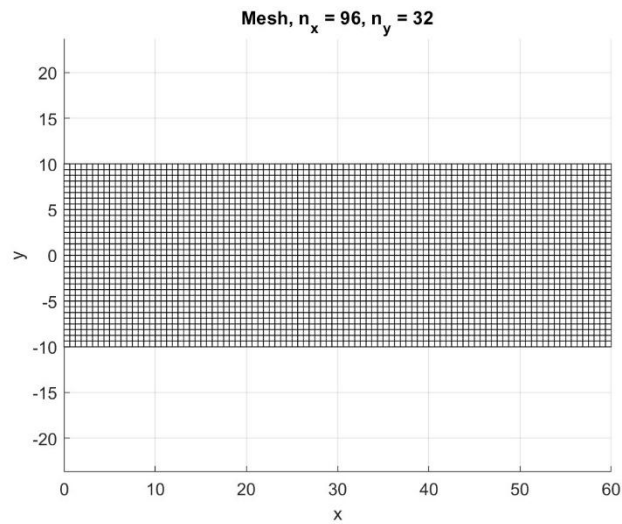
而在圖十八中我們將 FEM 解和正解的誤差以及每個粗網格 FEM 解和最細網格 FEM 解的誤差分別輸出並疊圖比較，可發現在 log-log 圖中，FEM 解和正解的誤差並不會呈現一直線，由表八也可發現每當 h 下降為原本的 $1/2$ ，每次的誤差會下降為原本的 0.34、0.41、0.60、0.81，下降幅度呈現趨於平緩的特性，當 h 越小時，誤差下降的幅度會變得越小，並無我們前面推導出的 $\text{Displacement Error} = O(h^2)$ 關係。

然而在每個粗網格 FEM 解和最細網格 FEM 解的誤差關係中，我們可發現其在 log-log 圖中呈現斜率大約為 2 的一直線，由表八可看出每當 h 下降為原本的 $1/2$ 時，每次的誤差大約下降為原本的 0.29、0.26、0.22，雖然下降比例與 0.25 有些許偏差，但可大致上看得出

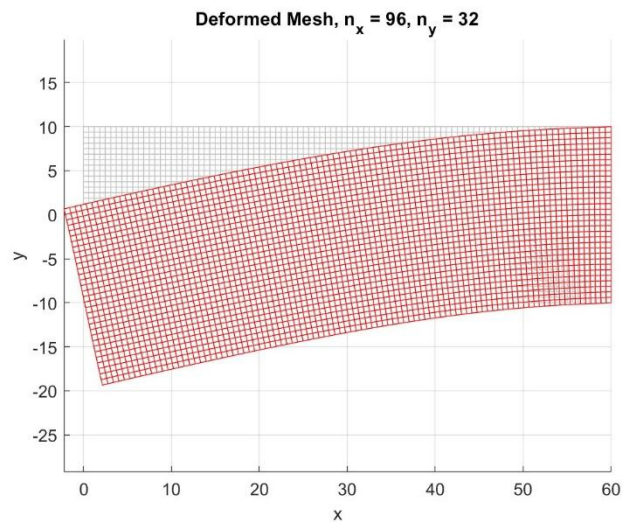
$\text{Displacement Error} = O(h^2)$ 的關係。

這邊推測會出現上面現象的原因為在分析 FEM 解和正解的誤差中，我在這邊用來計算正解的 Timoshenko beam solution 方法比較適合計算一

維樑模型，與 FEM 解的二維平面問題存在些許的模型差異，導致這邊在計算正解最大誤差時其中會包含離散誤差和模型差異誤差，而依照理論來說，當元素邊長 h 降至很小的時候其離散誤差會以二次方的速度快速收斂，但模型差異誤差並不會因為網格變細而消失，所以讓整體的正解最大誤差會有一種逐漸趨於平緩的現象。但當我們以最細網格當作參考，計算每個粗網格 FEM 解和最細網格 FEM 解的誤差時，由於兩者計算上是使用相同模型，所以上面講的模型差異誤差會消失，讓這邊表格中的最細網格參考誤差僅為離散誤差，便可看出上面推導的 $\text{Displacement Error} = O(h^2)$ 關係。



圖十九、最密網格的 xy 位置圖

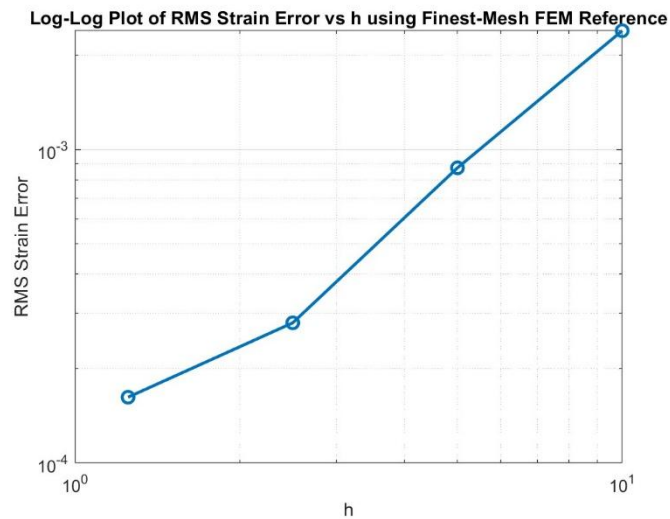


圖二十、最密網格受力時的 xy 位置圖

這邊我們也將網格最密時的受力以及非受力情況以最直觀的 xy 位置圖表示，若觀察 $x = 0$ 的位置可發現其中點在垂直的位移上與我們前面得到的數值相吻合，而在接近 $x = 1$ 處則因為定義的邊界條件讓元素的位移量非常少。此兩張圖讓我們非常直觀的觀察到了當一個懸臂量尾端受力時其元素的位移情形為如何。

表九、不同網格下的最細網格參考之應變 RMS 誤差

N_x	N_y	h (mm)	最細網格參考之應變 RMS 誤差
6	2	10	2.391×10^{-3}
12	4	5	8.738×10^{-4}
24	8	2.5	2.799×10^{-4}
48	16	1.25	1.621×10^{-4}
96	32	0.625	<i>NaN</i>



圖二十一、最細網格參考之應變 RMS 誤差與 h 之關係圖

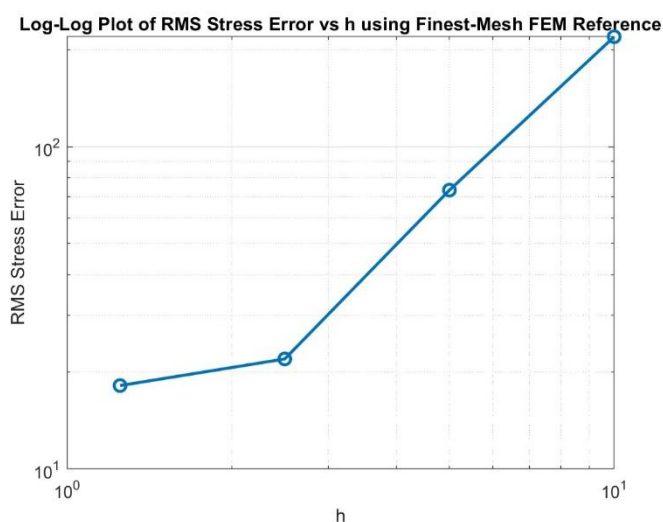
我們輸出了應變對於元素邊長 h 的關係圖，一樣參考前面的做法將最細網格所得之數值當作參考以計算誤差，這樣可以將重點聚焦在單純離散化之後的誤差現象，不會受到其他原因的所造成的誤差影響。而這邊之所以不像前面一樣使用最大值計算誤差的原因是由於應變的誤差容易受到局部位置的影響而放大，但若使用 RMS 計算誤差的話可以將整體的誤差進行平均，減少局部點控制整體誤差的機會。

這邊可看出在 log-log 圖中，最細網格參考之應變 RMS 誤差與元素邊長 h 大致上呈一斜率為 1 的直線，與前面理論推導的相符，即 $\text{strain error} = O(h)$ ，但由於還是有些許離群值影響會整體誤差的數值，再加上

計算應變會對位移進行微分，導致誤差的放大，這些不穩定的原因都讓應變誤差在圖中並非顯示為一完美的直線，在部分地方會有些許的偏差。

表十、不同網格下的最細網格參考之應力 RMS 誤差

N_x	N_y	h (mm)	最細網格參考之應力 RMS 誤差 (N/mm ²)
6	2	10	2.193×10^2
12	4	5	7.334×10^1
24	8	2.5	2.194×10^1
48	16	1.25	1.814×10^1
96	32	0.625	<i>NaN</i>



圖二十二、最細網格參考之應力 RMS 誤差與 h 之關係圖

這邊一樣輸出最細網格參考之應力 RMS 誤差與元素邊長 h 的關係圖，可發現其與應變大致上呈一樣的趨勢，與理論上的 $\text{stress error} = O(h)$ 大致上相符。

Appendices

這邊使用 AI 協助程式上的撰寫，並且嚴格要求 AI 「只輸出程式碼給我就好不要有任何文字敘述」。

AI 對話連結：<https://chatgpt.com/share/6a26fb07-1990-83a6-acb3-9d6f1a86e24e>

B12502027 Q2 b (ii)

Prompt （按照指令順序依次全部附上）：

1.
$$\begin{bmatrix} b & c & 0 & \cdots & 0 & 0 & 0 \\ a & b & c & \cdots & 0 & 0 & 0 \\ 0 & a & b & \ddots & 0 & 0 & 0 \\ & \vdots & & \ddots & \vdots & & \\ 0 & 0 & 0 & & b & c & 0 \\ 0 & 0 & 0 & \cdots & a & b & c \\ 0 & 0 & 0 & & 0 & a & b \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-4} \\ y_{N-3} \\ y_{N-2} \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{N-4} \\ x_{N-3} \\ x_{N-2} - 2c \end{bmatrix}$$

$$a=(1/(\Delta x^2))-2/\Delta x, b=(4-2/(\Delta x^2)), c=(1/(\Delta x^2))+2/\Delta x$$

以上是我求得之線性方程組，幫我用 MATLAB 寫一個程式碼，利用高斯消去法解這個線性方程組（Don't use the backslash function (\) in Matlab to solve the set of linear equations），我的 Δx 要是一個變數讓我觀察在不同 Δx 的情況下誤差的表現，只輸出程式碼給我就好不要有任何文字敘述

2. 幫我把得到的解和解析解的值也表示出來，不要只有誤差
3. 也要輸出每組不同 dx 計算之後的殘值
4. 將 $dx=1/512$ 輸出一個圖，橫軸為 x ，縱軸為誤差
5. 輸出所有不同 dx 時的這個圖
6. 幫我輸出一張疊圖裏面包含解析解以及不同 dx 下算出的解
7. 畫線就好不需要標出每個點，
8. 輸出所有不同 dx 時的這個圖時幫我調一下縱軸讓蝶圖不會都黏在一起
9. 將 $dx=1/64$ 輸出一個圖，橫軸為 x ，縱軸為誤差
10. 將點表示出來

（輸入完以上所有 Prompt 後 AI 都僅生成程式碼給我而已）

最終程式碼（報告中有些圖是利用中間幾次 Prompt 生成的程式碼所得，並非由次最終程式碼輸出）：

```
clear; clc; close all;
```

```
dx_list = [1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512];
```

```
error_list = zeros(size(dx_list));
```

```
residual_list = zeros(size(dx_list));
```

```
figure;
```

```
hold on;
```

```

grid on;

x_exact_plot = linspace(0, 1, 1000);
y_exact_plot = exactSolution(x_exact_plot);

plot(x_exact_plot, y_exact_plot, 'k-', 'LineWidth', 2.5, ...
     'DisplayName', 'Exact Solution');

for k = 1:length(dx_list)

    dx = dx_list(k);

    N = round(1/dx) + 1;
    n = N - 2;

    x = (1:n)' * dx;

    a = 1/(dx^2) - 2/dx;
    b = 4 - 2/(dx^2);
    c = 1/(dx^2) + 2/dx;

    A = zeros(n, n);

    for i = 1:n
        A(i,i) = b;

        if i > 1
            A(i,i-1) = a;
        end

        if i < n
            A(i,i+1) = c;
        end
    end

    rhs = x;
    rhs(end) = rhs(end) - 2*c;

```

```

y_numerical = gaussianElimination(A, rhs);

y_exact = exactSolution(x);
abs_error = abs(y_numerical - y_exact);

residual = norm(A*y_numerical - rhs, inf);
error = norm(abs_error, inf);

residual_list(k) = residual;
error_list(k) = error;

fprintf('\n=====\\n');
fprintf('dx = %.8f\\n', dx);
fprintf('Residual = %.12e\\n', residual);
fprintf('Maximum Error = %.12e\\n', error);
fprintf('=====\\n');

result_table = table(x, y_numerical, y_exact, abs_error, ...
    'VariableNames', {'x', 'Numerical_y', 'Exact_y', 'Absolute_Error'});

disp(result_table);

x_with_bc = [0; x; 1];
y_with_bc = [0; y_numerical; 2];

plot(x_with_bc, y_with_bc, '-o', ...
    'DisplayName', ['Numerical, \\Deltax = ', num2str(dx)]);

end

xlabel('x');
ylabel('y');
title('Exact Solution and Numerical Solutions for Different \\Deltax');
legend('show', 'Location', 'best');

summary_table = table(dx_list, error_list, residual_list, ...
    'VariableNames', {'dx', 'Max_Error', 'Residual'});

```

```
fprintf('\n=====\\n');  
fprintf('Summary Table\\n');  
fprintf('=====\\n');
```

```
disp(summary_table);
```

```
figure;  
hold on;  
grid on;
```

```
for k = 1:length(dx_list)
```

```
    dx = dx_list(k);
```

```
    N = round(1/dx) + 1;
```

```
    n = N - 2;
```

```
    x = (1:n)' * dx;
```

```
    a = 1/(dx^2) - 2/dx;
```

```
    b = 4 - 2/(dx^2);
```

```
    c = 1/(dx^2) + 2/dx;
```

```
    A = zeros(n, n);
```

```
    for i = 1:n
```

```
        A(i,i) = b;
```

```
        if i > 1
```

```
            A(i,i-1) = a;
```

```
        end
```

```
        if i < n
```

```
            A(i,i+1) = c;
```

```
        end
```

```
    end
```

```
    rhs = x;
```



```

    rhs(end) = rhs(end) - 2*c;

    y_numerical = gaussianElimination(A, rhs);
    y_exact = exactSolution(x);

    abs_error = abs(y_numerical - y_exact);

    plot(x, abs_error, '-o', ...
         'DisplayName', ['\Deltax = ', num2str(dx)]);

end

xlabel('x');
ylabel('Error');
title('Error vs x for Different \Deltax');
legend('show', 'Location', 'best');

figure;
loglog(dx_list, error_list, '-o');
grid on;
xlabel('\Deltax');
ylabel('Maximum Error');
title('Maximum Error vs \Deltax');

figure;
loglog(dx_list, residual_list, '-o');
grid on;
xlabel('\Deltax');
ylabel('Residual');
title('Residual vs \Deltax');

function y = gaussianElimination(A, b)

    n = length(b);
    Aug = [A b];

    for k = 1:n-1

```

```

[~, pivotRow] = max(abs(Aug(k:n,k)));
pivotRow = pivotRow + k - 1;

if pivotRow ~= k
    temp = Aug(k,:);
    Aug(k,:) = Aug(pivotRow,:);
    Aug(pivotRow,:) = temp;
end

for i = k+1:n
    factor = Aug(i,k) / Aug(k,k);
    Aug(i,k:end) = Aug(i,k:end) - factor * Aug(k,k:end);
end

end

y = zeros(n,1);

for i = n:-1:1
    y(i) = (Aug(i,end) - Aug(i,i+1:n) * y(i+1:n)) / Aug(i,i);
end

end

function y = exactSolution(x)

    C1 = 1/4;
    C2 = 2*exp(2) - 1/4;

    y = (C1 + C2*x).*exp(-2*x) + (x - 1)/4;

end

```

B12502027 Q2 c (iii)

Prompt （按照指令順序依次全部附上）：

$$1. \quad y_n = u_{1,n} + \frac{2-u_{1,N}}{v_{1,N}} v_{1,n}, \quad n = 0, 1, 2, \dots, N \quad \begin{cases} u_{1,n+1} = u_{1,n} + \Delta x u_{2,n} \\ u_{2,n+1} = u_{2,n} + \Delta x (-4u_{2,n} - 4u_{1,n} + x) \end{cases} \quad \begin{cases} v_{1,n+1} = v_{1,n} + \Delta x v_{2,n} \\ v_{2,n+1} = v_{2,n} + \Delta x (-4v_{2,n} - 4v_{1,n}) \end{cases}$$

以上是我將 Explicit Euler Method 代入兩個 IVPs 後求得的解，幫我用 MATLAB 寫一個程式碼計算，這邊 u_1 為非齊次解， v_1 為齊次解，我的 Δx 要是一個變數讓我觀察在不同 Δx 的情況下的表現，只輸出程式碼給我就不好不要有任何文字敘述

2. 不用輸出殘值，error 對 x 的圖幫我把每個 dx 都分開輸出，且 error 對 x 的圖幫我把點也都標出來，然後我有 $dx_list = [1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512]$;
3. 我是說 error 對 x 的圖幫我把每個 dx 都分開輸出，y 對 x 的圖還是一樣疊圖
4. 將 $dx=1/512$ 輸出一個圖，橫軸為 x，縱軸為誤差
(輸入完以上所有 Prompt 後 AI 都僅生成程式碼給我而已)

最終程式碼 (報告中有些圖是利用中間幾次 Prompt 生成的程式碼所得，並非由次最終程式碼輸出) :

```
clear; clc; close all;
```

```
dx_list = [1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512];
```

```
error_list = zeros(size(dx_list));
```

```
residual_list = zeros(size(dx_list));
```

```
figure;
```

```
hold on;
```

```
grid on;
```

```
x_exact_plot = linspace(0, 1, 1000)';
```

```
y_exact_plot = exactSolution(x_exact_plot);
```

```
plot(x_exact_plot, y_exact_plot, 'k-', 'LineWidth', 2.5, ...  
     'DisplayName', 'Exact Solution');
```

```
for k = 1:length(dx_list)
```

```
    dx = dx_list(k);
```

```
    N = round(1/dx);
```

```
    x = (0:N)' * dx;
```

```

u1 = zeros(N+1, 1);
u2 = zeros(N+1, 1);
v1 = zeros(N+1, 1);
v2 = zeros(N+1, 1);

u1(1) = 0;
u2(1) = 0;

v1(1) = 0;
v2(1) = 1;

for n = 1:N
    u1(n+1) = u1(n) + dx*u2(n);
    u2(n+1) = u2(n) + dx*(-4*u2(n) - 4*u1(n) + x(n));

    v1(n+1) = v1(n) + dx*v2(n);
    v2(n+1) = v2(n) + dx*(-4*v2(n) - 4*v1(n));
end

theta = (2 - u1(end)) / v1(end);

y_numerical = u1 + theta*v1;
y_exact = exactSolution(x);

abs_error = abs(y_numerical - y_exact);
max_error = norm(abs_error, inf);

residual = abs(y_numerical(end) - 2);

error_list(k) = max_error;
residual_list(k) = residual;

fprintf('\n=====\\n');
fprintf('dx = %.8f\\n', dx);
fprintf('theta = %.12e\\n', theta);
fprintf('Residual = %.12e\\n', residual);
fprintf('Maximum Error = %.12e\\n', max_error);

```

```

fprintf('=====\n');

result_table = table(x, u1, v1, y_numerical, y_exact, abs_error, ...
    'VariableNames', {'x', 'u1_nonhomogeneous', 'v1_homogeneous', ...
    'Numerical_y', 'Exact_y', 'Absolute_Error'});

disp(result_table);

plot(x, y_numerical, '-', 'LineWidth', 1.5, ...
    'DisplayName', ['Numerical, \Deltax = ', num2str(dx)]);

end

xlabel('x');
ylabel('y');
title('Exact Solution and Explicit Euler Shooting Solutions');
legend('show', 'Location', 'best');

summary_table = table(dx_list, error_list, residual_list, ...
    'VariableNames', {'dx', 'Max_Error', 'Residual'});

fprintf("\n=====\n");
fprintf('Summary Table\n');
fprintf('=====\n');

disp(summary_table);

figure;
hold on;
grid on;

for k = 1:length(dx_list)

    dx = dx_list(k);
    N = round(1/dx);

    x = (0:N)' * dx;

```

```

u1 = zeros(N+1, 1);
u2 = zeros(N+1, 1);
v1 = zeros(N+1, 1);
v2 = zeros(N+1, 1);

u1(1) = 0;
u2(1) = 0;

v1(1) = 0;
v2(1) = 1;

for n = 1:N
    u1(n+1) = u1(n) + dx*u2(n);
    u2(n+1) = u2(n) + dx*(-4*u2(n) - 4*u1(n) + x(n));

    v1(n+1) = v1(n) + dx*v2(n);
    v2(n+1) = v2(n) + dx*(-4*v2(n) - 4*v1(n));
end

theta = (2 - u1(end)) / v1(end);

y_numerical = u1 + theta*v1;
y_exact = exactSolution(x);

abs_error = abs(y_numerical - y_exact);

plot(x, abs_error, '-', 'LineWidth', 1.5, ...
     'DisplayName', ['\Delta x = ', num2str(dx)]);

end

xlabel('x');
ylabel('Error');
title('Error vs x for Different \Delta x');
legend('show', 'Location', 'best');

figure;
loglog(dx_list, error_list, '-o', 'LineWidth', 1.5);

```

```

grid on;
xlabel('\Delta x');
ylabel('Maximum Error');
title('Maximum Error vs \Delta x');

figure;
loglog(dx_list, residual_list, '-o', 'LineWidth', 1.5);
grid on;
xlabel('\Delta x');
ylabel('Residual');
title('Residual vs \Delta x');

```

```
function y = exactSolution(x)
```

```
C1 = 1/4;
```

```
C2 = 2*exp(2) - 1/4;
```

```
y = (C1 + C2*x).*exp(-2*x) + (x - 1)/4;
```

```
end
```

B12502027 Q2 c (iv)

Prompt （按照指令順序依次全部附上）：

$$\begin{aligned}
 &\text{令 } M_1 = I - \frac{\Delta x}{2} A, \quad M_2 = I + \frac{\Delta x}{2} A \\
 &\rightarrow \begin{cases} M_1 u_{n+1} = M_2 u_n + \frac{\Delta x}{2} (B(x_n) + B(x_{n+1})) \\ M_1 v_{n+1} = M_2 v_n \end{cases} \quad y_n = u_{1,n} + \frac{2 - u_{1,N}}{v_{1,N}} v_{1,n}, \quad n = 0, 1, 2, \dots, N
 \end{aligned}$$

1. 以

上是我將 Crank–Nicolson Method 代入兩個 IVPs 後求得的解，幫我用 MATLAB 寫一個程式碼計算，我的 Δx 要是一個變數讓我觀察在不同 Δx 的情況下的誤差的表現，只輸出程式碼給我就好不要有任何文字敘述（輸入完以上所有 Prompt 後 AI 都僅生成程式碼給我而已）

最終程式碼（報告中有些圖是利用中間幾次 Prompt 生成的程式碼所得，並非由次最終程式碼輸出）：

```
clear; clc; close all;
```

```
dx_list = [1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512];
```

```

error_list = zeros(size(dx_list));

A = [0 1;
     -4 -4];

fig_y = figure;
hold on;
grid on;

x_exact_plot = linspace(0, 1, 1000)';
y_exact_plot = exactSolution(x_exact_plot);

plot(x_exact_plot, y_exact_plot, 'k-', 'LineWidth', 2.5, ...
     'DisplayName', 'Exact Solution');

for k = 1:length(dx_list)

    dx = dx_list(k);
    N = round(1/dx);

    x = (0:N)' * dx;

    u = zeros(2, N+1);
    v = zeros(2, N+1);

    u(:,1) = [0; 0];
    v(:,1) = [0; 1];

    M1 = eye(2) - dx/2 * A;
    M2 = eye(2) + dx/2 * A;

    for n = 1:N

        Bn    = [0; x(n)];
        Bnp1 = [0; x(n+1)];

        rhs_u = M2*u(:,n) + dx/2 * (Bn + Bnp1);

```



```

    rhs_v = M2*v(:,n);

    u(:,n+1) = solve2by2(M1, rhs_u);
    v(:,n+1) = solve2by2(M1, rhs_v);

end

u1 = u(1,:);
v1 = v(1,:);

theta = (2 - u1(end)) / v1(end);

y_numerical = u1 + theta*v1;
y_exact = exactSolution(x);

abs_error = abs(y_numerical - y_exact);
max_error = norm(abs_error, inf);

error_list(k) = max_error;

fprintf('\n=====\\n');
fprintf('dx = %.8f\\n', dx);
fprintf('theta = %.12e\\n', theta);
fprintf('Maximum Error = %.12e\\n', max_error);
fprintf('=====\\n');

result_table = table(x, u1, v1, y_numerical, y_exact, abs_error, ...
    'VariableNames', {'x', 'u1_nonhomogeneous', 'v1_homogeneous', ...
    'Numerical_y', 'Exact_y', 'Absolute_Error'});

disp(result_table);

figure(fig_y);
plot(x, y_numerical, '-', 'LineWidth', 1.5, ...
    'DisplayName', ['Numerical, \\Deltax = ', num2str(dx)]);

figure;
plot(x, abs_error, '-o', 'LineWidth', 1.5, 'MarkerSize', 5);

```

```

        grid on;
        xlabel('x');
        ylabel('Error');
        title(['Error vs x when \Deltax = ', num2str(dx)]);

end

figure(fig_y);
xlabel('x');
ylabel('y');
title('Exact Solution and Crank-Nicolson Shooting Solutions');
legend('show', 'Location', 'best');

summary_table = table(dx_list, error_list, ...
    'VariableNames', {'dx', 'Maximum_Error'});

fprintf('\n=====\\n');
fprintf('Summary Table\\n');
fprintf('=====\\n');

disp(summary_table);

figure;
loglog(dx_list, error_list, '-o', 'LineWidth', 1.5, 'MarkerSize', 6);
grid on;
xlabel('\Deltax');
ylabel('Maximum Error');
title('Maximum Error vs \Deltax');

function x = solve2by2(A, b)

    detA = A(1,1)*A(2,2) - A(1,2)*A(2,1);

    x = zeros(2,1);
    x(1) = ( b(1)*A(2,2) - A(1,2)*b(2)) / detA;
    x(2) = ( A(1,1)*b(2) - b(1)*A(2,1)) / detA;

```

end

```
function y = exactSolution(x)
```

```
    C1 = 1/4;
```

```
    C2 = 2*exp(2) - 1/4;
```

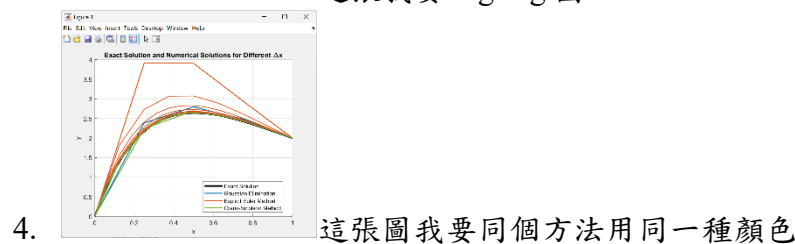
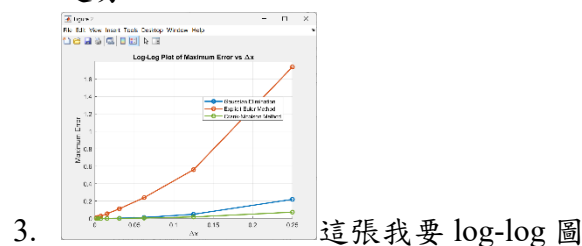
```
    y = (C1 + C2*x).*exp(-2*x) + (x - 1)/4;
```

end

B12502027 Q2 dicussion

Prompt （按照指令順序依次全部附上）：

1. 寫一個 MATLAB 程式碼，把三種方法 Gaussian Elimination、Explicit Euler Method、Crank–Nicolson Method 的 Maximum Error 與 Δx 關係圖、解析解以及不同 Δx 下數值解的比較圖疊圖，只要程式碼
2. Maximum Error 與 Δx 關係圖出 log-log 圖，解析解以及不同 Δx 下數值解的比較圖把相同方法的以同樣顏色表示，然後把三張圖疊起來，我想看收斂趨勢



（輸入完以上所有 Prompt 後 AI 都僅生成程式碼給我而已）

最終程式碼（報告中有些圖是利用中間幾次 Prompt 生成的程式碼所得，並非由次最終程式碼輸出）：

```
clear; clc; close all;
```

```
dx_list = [1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512];
```

```

err_GE = zeros(size(dx_list));
err_EE = zeros(size(dx_list));
err_CN = zeros(size(dx_list));

color_GE = [0.0000 0.4470 0.7410];
color_EE = [0.8500 0.3250 0.0980];
color_CN = [0.4660 0.6740 0.1880];

x_exact_plot = linspace(0,1,2000)';
y_exact_plot = exactSolution(x_exact_plot);

figure(1);
hold on; grid on;
plot(x_exact_plot, y_exact_plot, 'k-', 'LineWidth', 3, ...
     'DisplayName', 'Exact Solution');

for k = 1:length(dx_list)

    dx = dx_list(k);

    [x_GE, y_GE] = GaussianEliminationMethod(dx);
    [x_EE, y_EE] = ExplicitEulerMethod(dx);
    [x_CN, y_CN] = CrankNicolsonMethod(dx);

    err_GE(k) = norm(y_GE - exactSolution(x_GE), inf);
    err_EE(k) = norm(y_EE - exactSolution(x_EE), inf);
    err_CN(k) = norm(y_CN - exactSolution(x_CN), inf);

    if k == 1
        plot(x_GE, y_GE, '-', 'Color', color_GE, 'LineWidth', 1.5, ...
             'DisplayName', 'Gaussian Elimination');
        plot(x_EE, y_EE, '-', 'Color', color_EE, 'LineWidth', 1.5, ...
             'DisplayName', 'Explicit Euler Method');
        plot(x_CN, y_CN, '-', 'Color', color_CN, 'LineWidth', 1.5, ...
             'DisplayName', 'Crank-Nicolson Method');
    else
        plot(x_GE, y_GE, '-', 'Color', color_GE, 'LineWidth', 1.5, ...
             'HandleVisibility', 'off');
    end
end

```

```

        plot(x_EE, y_EE, '-', 'Color', color_EE, 'LineWidth', 1.5, ...
              'HandleVisibility', 'off');
        plot(x_CN, y_CN, '-', 'Color', color_CN, 'LineWidth', 1.5, ...
              'HandleVisibility', 'off');
    end

end

xlabel('x');
ylabel('y');
title('Exact Solution and Numerical Solutions for Different \Delta x');
legend('show', 'Location', 'best');


figure(2);
hold on; grid on;

loglog(dx_list, err_GE, '-o', 'Color', color_GE, ...
        'LineWidth', 1.8, 'MarkerSize', 6, ...
        'DisplayName', 'Gaussian Elimination');

loglog(dx_list, err_EE, '-o', 'Color', color_EE, ...
        'LineWidth', 1.8, 'MarkerSize', 6, ...
        'DisplayName', 'Explicit Euler Method');

loglog(dx_list, err_CN, '-o', 'Color', color_CN, ...
        'LineWidth', 1.8, 'MarkerSize', 6, ...
        'DisplayName', 'Crank-Nicolson Method');

set(gca, 'XScale', 'log');
set(gca, 'YScale', 'log');

xlabel('\Delta x');
ylabel('Maximum Error');
title('Log-Log Plot of Maximum Error vs \Delta x');
legend('show', 'Location', 'best');
xticks(dx_list);
xticklabels({'1/4','1/8','1/16','1/32','1/64','1/128','1/256','1/512'});

```

```
summary_table = table(dx_list', err_GE', err_EE', err_CN', ...
    'VariableNames', {'dx', 'Gaussian_Elimination_Error', ...
    'Explicit_Euler_Error', 'Crank_Nicolson_Error'});
```

```
disp(summary_table);
```

```
function [x_all, y_all] = GaussianEliminationMethod(dx)
```

```
    N = round(1/dx) + 1;
```

```
    n = N - 2;
```

```
    x = (1:n)' * dx;
```

```
    a = 1/(dx^2) - 2/dx;
```

```
    b = 4 - 2/(dx^2);
```

```
    c = 1/(dx^2) + 2/dx;
```

```
    A = zeros(n, n);
```

```
    for i = 1:n
```

```
        A(i,i) = b;
```

```
        if i > 1
```

```
            A(i,i-1) = a;
```

```
        end
```

```
        if i < n
```

```
            A(i,i+1) = c;
```

```
        end
```

```
    end
```

```
    rhs = x;
```

```
    rhs(end) = rhs(end) - 2*c;
```

```
    y = gaussianElimination(A, rhs);
```

```
    x_all = [0; x; 1];
```

```
    y_all = [0; y; 2];
```

end

function [x, y] = ExplicitEulerMethod(dx)

N = round(1/dx);

x = (0:N)' * dx;

u1 = zeros(N+1,1);

u2 = zeros(N+1,1);

v1 = zeros(N+1,1);

v2 = zeros(N+1,1);

u1(1) = 0;

u2(1) = 0;

v1(1) = 0;

v2(1) = 1;

for n = 1:N

u1(n+1) = u1(n) + dx*u2(n);

u2(n+1) = u2(n) + dx*(-4*u2(n) - 4*u1(n) + x(n));

v1(n+1) = v1(n) + dx*v2(n);

v2(n+1) = v2(n) + dx*(-4*v2(n) - 4*v1(n));

end

theta = (2 - u1(end)) / v1(end);

y = u1 + theta*v1;

end

function [x, y] = CrankNicolsonMethod(dx)

N = round(1/dx);

x = (0:N)' * dx;

```

A = [0 1; -4 -4];
M1 = eye(2) - dx/2*A;
M2 = eye(2) + dx/2*A;

u = zeros(2, N+1);
v = zeros(2, N+1);

u(:,1) = [0; 0];
v(:,1) = [0; 1];

for n = 1:N
    Bn    = [0; x(n)];
    Bnp1 = [0; x(n+1)];

    rhs_u = M2*u(:,n) + dx/2*(Bn + Bnp1);
    rhs_v = M2*v(:,n);

    u(:,n+1) = solve2by2(M1, rhs_u);
    v(:,n+1) = solve2by2(M1, rhs_v);
end

u1 = u(1,:);
v1 = v(1,:);

theta = (2 - u1(end)) / v1(end);
y = u1 + theta*v1;

```

end

function y = gaussianElimination(A, b)

```

n = length(b);
Aug = [A b];

for k = 1:n-1
    [~, pivotRow] = max(abs(Aug(k:n,k)));
    pivotRow = pivotRow + k - 1;

```



```

    if pivotRow ~= k
        temp = Aug(k,:);
        Aug(k,:) = Aug(pivotRow,:);
        Aug(pivotRow,:) = temp;
    end

    for i = k+1:n
        factor = Aug(i,k) / Aug(k,k);
        Aug(i,k:end) = Aug(i,k:end) - factor * Aug(k,k:end);
    end
end

y = zeros(n,1);

for i = n:-1:1
    y(i) = (Aug(i,end) - Aug(i,i+1:n)*y(i+1:n)) / Aug(i,i);
end

end

```

```

function x = solve2by2(A, b)

```

```

    detA = A(1,1)*A(2,2) - A(1,2)*A(2,1);

    x = zeros(2,1);
    x(1) = (b(1)*A(2,2) - A(1,2)*b(2)) / detA;
    x(2) = (A(1,1)*b(2) - b(1)*A(2,1)) / detA;

end

```

```

function y = exactSolution(x)

```

```

    C1 = 1/4;
    C2 = 2*exp(2) - 1/4;
    y = (C1 + C2*x).*exp(-2*x) + (x - 1)/4;

```

end

B12502027 Q3 Q4

Prompt （按照指令順序依次全部附上）：

I. **Problem description:** Consider the linear elasticity of a cantilever beam with a length of L , a height of $2a$, and a width of b as shown in figure 1.

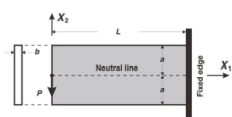


Figure 1: Schematics of the cantilever beam.

IV. **Finite element approximation:** For the numerical computation, the parameters are chosen as: $P = 8 \text{ kN}$, $L = 60 \text{ mm}$, $a = 10 \text{ mm}$, $b = 1 \text{ mm}$, $E = 100 \text{ GPa}$, $\nu = 0.3$. The undeformed beam is then modeled as an assemblage of identical 4-node-quadrilateral elements. When the end load P is applied, the beam deforms as shown in figure 2.

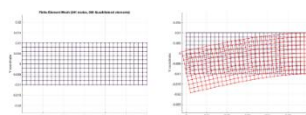
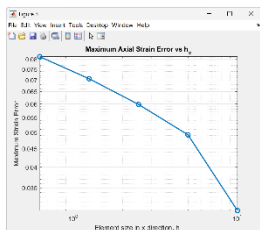


Figure 2: Mesh and Deformation (bending) of the cantilever beam.

以上是

我們的題目，寫一個 MATLAB 程式碼，幫我利用有限元素法模擬，我的元素尺寸 h 要是一個可調的變數，讓我觀察在不同 h 時的誤差表現，只要程式碼不要解釋

- 元素尺寸 h 的意思時我可以設定分別在 x 以及 y 方向上的網格數
- 也要輸出 exact solution，這樣我才能觀察數值解跟正解之間的誤差
- 把 Exact Solution and Numerical Deflection 在不同的網格數量時跌圖
- exact solution 錯了，將 exact solution 改成 Timoshenko beam 形式並重新輸出一個程式碼
- 將 finest-mesh reference error 也納入考量
- $nx_list = [6, 12, 24, 48, 96]$; $ny_list = [2, 4, 8, 16, 32]$; 還要輸出 mesh 的圖
- 現在不用輸出 mesh 的圖了，但幫我輸出應變的解析解以及數值解誤差



- 這張出 log-log 圖

- 改成不同 h 下的 max strain error
- 我是指整體程式碼改成不同 h 下的 max strain error
- 做不同 h 的 strain error 對 h 的 log-log 圖。
- 做不同 h 的 stress error 對 h 的 log-log 圖。
- 做不同 h 的 strain error 對 h 的 log-log 圖的部分以 finest-mesh FEM strain 當參考計算誤差
- 改用 RMS error
- 將應力的也顯示出來
- (輸入完以上所有 Prompt 後 AI 都僅生成程式碼給我而已)

最終程式碼（報告中有些圖是利用中間幾次 Prompt 生成的程式碼所得，並非由次最終程式碼輸出）：

```
clear; clc; close all;
```

```
P = 8000;
```

```
L = 60;
```

```
a = 10;
```

```
b = 1;
```

```
E = 100000;
```

```
nu = 0.3;
```

```
nx_list = [6, 12, 24, 48, 96];
```

```
ny_list = [2, 4, 8, 16, 32];
```

```
rms_strain_error_ref_list = zeros(size(nx_list));
```

```
rms_stress_error_ref_list = zeros(size(nx_list));
```

```
hx_list = zeros(size(nx_list));
```

```
hy_list = zeros(size(nx_list));
```

```
h_list = zeros(size(nx_list));
```

```
D = E/(1-nu^2) * [1 nu 0;
```

```
                  nu 1 0;
```

```
                  0 0 (1-nu)/2];
```

```
all_x_strain = cell(length(nx_list),1);
```

```
all_y_strain = cell(length(nx_list),1);
```

```
all_eps_num = cell(length(nx_list),1);
```

```
all_sig_num = cell(length(nx_list),1);
```

```
for m = 1:length(nx_list)
```

```
    nx = nx_list(m);
```

```
    ny = ny_list(m);
```

```
    hx = L / nx;
```

```
    hy = 2*a / ny;
```

```
    h = max(hx, hy);
```

```

hx_list(m) = hx;
hy_list(m) = hy;
h_list(m) = h;

[node, elem] = generateMesh(L, a, nx, ny);

numNode = size(node,1);
numElem = size(elem,1);
numDOF = 2*numNode;

K = zeros(numDOF, numDOF);
F = zeros(numDOF, 1);

for e = 1:numElem

    elemNode = elem(e,:);
    coord = node(elemNode,:);

    Ke = Q4ElementStiffness(coord, D, b);

    dof = zeros(8,1);
    for i = 1:4
        dof(2*i-1) = 2*elemNode(i)-1;
        dof(2*i) = 2*elemNode(i);
    end

    K(dof,dof) = K(dof,dof) + Ke;

end

leftNodes = find(abs(node(:,1)) < 1e-12);
[~, idxSort] = sort(node(leftNodes,2));
leftNodes = leftNodes(idxSort);

for i = 1:length(leftNodes)-1

    n1 = leftNodes(i);

```

```

n2 = leftNodes(i+1);

edgeLength = abs(node(n2,2) - node(n1,2));

F(2*n1) = F(2*n1) - P * edgeLength/(2*(2*a));
F(2*n2) = F(2*n2) - P * edgeLength/(2*(2*a));

end

fixedNodes = find(abs(node(:,1)-L) < 1e-12);

fixedDOF = zeros(2*length(fixedNodes),1);
for i = 1:length(fixedNodes)
    fixedDOF(2*i-1) = 2*fixedNodes(i)-1;
    fixedDOF(2*i)   = 2*fixedNodes(i);
end

allDOF = (1:numDOF)';
freeDOF = setdiff(allDOF, fixedDOF);

U = zeros(numDOF,1);
U(freeDOF) = K(freeDOF,freeDOF) \ F(freeDOF);

[x_strain, y_strain, eps_num, sig_num] =
computeElementStrainStressAtCenter(node, elem, U, D);

all_x_strain{m} = x_strain;
all_y_strain{m} = y_strain;
all_eps_num{m}  = eps_num;
all_sig_num{m}  = sig_num;

end

x_ref = all_x_strain{end};
y_ref = all_y_strain{end};
eps_ref = all_eps_num{end};
sig_ref = all_sig_num{end};

```

```
F_eps_ref = scatteredInterpolant(x_ref, y_ref, eps_ref, 'linear', 'nearest');
```

```
F_sig_ref = scatteredInterpolant(x_ref, y_ref, sig_ref, 'linear', 'nearest');
```

```
for m = 1:length(nx_list)
```

```
    x_strain = all_x_strain{m};
```

```
    y_strain = all_y_strain{m};
```

```
    eps_num = all_eps_num{m};
```

```
    sig_num = all_sig_num{m};
```

```
    eps_ref_interp = F_eps_ref(x_strain, y_strain);
```

```
    sig_ref_interp = F_sig_ref(x_strain, y_strain);
```

```
    eps_error_ref = eps_num - eps_ref_interp;
```

```
    sig_error_ref = sig_num - sig_ref_interp;
```

```
    rms_strain_error_ref_list(m) = sqrt(mean(eps_error_ref.^2));
```

```
    rms_stress_error_ref_list(m) = sqrt(mean(sig_error_ref.^2));
```

```
    if m == length(nx_list)
```

```
        rms_strain_error_ref_list(m) = NaN;
```

```
        rms_stress_error_ref_list(m) = NaN;
```

```
    end
```

```
    fprintf('\n=====\\n');
```

```
    fprintf('nx = %d, ny = %d\\n', nx_list(m), ny_list(m));
```

```
    fprintf('hx = %.12e\\n', hx_list(m));
```

```
    fprintf('hy = %.12e\\n', hy_list(m));
```

```
    fprintf('h = %.12e\\n', h_list(m));
```

```
    if m ~= length(nx_list)
```

```
        fprintf('RMS Strain Error compared with finest mesh = %.12e\\n', ...
```

```
            rms_strain_error_ref_list(m));
```

```
        fprintf('RMS Stress Error compared with finest mesh = %.12e\\n', ...
```

```
            rms_stress_error_ref_list(m));
```

```
    else
```

```
        fprintf('RMS Strain Error compared with finest mesh = NaN\\n');
```

```
        fprintf('RMS Stress Error compared with finest mesh = NaN\\n');
```

```

end

fprintf('=====\n');

strain_stress_table = table(x_strain, y_strain, ...
    eps_num, eps_ref_interp, abs(eps_error_ref), ...
    sig_num, sig_ref_interp, abs(sig_error_ref), ...
    'VariableNames', {'x', 'y', ...
    'Numerical_Axial_Strain', ...
    'Finest_Mesh_Reference_Strain', ...
    'Absolute_Strain_Error_Reference', ...
    'Numerical_Axial_Stress', ...
    'Finest_Mesh_Reference_Stress', ...
    'Absolute_Stress_Error_Reference'});

disp(strain_stress_table);

end

result_table = table(nx_list', ny_list', hx_list', hy_list', h_list', ...
    rms_strain_error_ref_list', rms_stress_error_ref_list', ...
    'VariableNames', {'nx', 'ny', 'hx', 'hy', 'h', ...
    'RMS_Strain_Error_Finest_Mesh_Reference', ...
    'RMS_Stress_Error_Finest_Mesh_Reference'});

disp(result_table);

figure;
loglog(h_list(1:end-1), rms_strain_error_ref_list(1:end-1), '-o', ...
    'LineWidth', 1.8, 'MarkerSize', 7);
grid on;
set(gca, 'XScale', 'log');
set(gca, 'YScale', 'log');
xlabel('h');
ylabel('RMS Strain Error');
title('Log-Log Plot of RMS Strain Error vs h using Finest-Mesh FEM Reference');

figure;

```

```

loglog(h_list(1:end-1), rms_stress_error_ref_list(1:end-1), '-o', ...
       'LineWidth', 1.8, 'MarkerSize', 7);
grid on;
set(gca, 'XScale', 'log');
set(gca, 'YScale', 'log');
xlabel('h');
ylabel('RMS Stress Error');
title('Log-Log Plot of RMS Stress Error vs h using Finest-Mesh FEM Reference');

```

```

figure;
hold on; grid on;

```

```

loglog(h_list(1:end-1), rms_strain_error_ref_list(1:end-1), '-o', ...
       'LineWidth', 1.8, 'MarkerSize', 7, ...
       'DisplayName', 'RMS Strain Error');

```

```

loglog(h_list(1:end-1), rms_stress_error_ref_list(1:end-1), '-s', ...
       'LineWidth', 1.8, 'MarkerSize', 7, ...
       'DisplayName', 'RMS Stress Error');

```

```

set(gca, 'XScale', 'log');
set(gca, 'YScale', 'log');
xlabel('h');
ylabel('RMS Error');
title('Log-Log Plot of RMS Strain and Stress Error vs h');
legend('show', 'Location', 'best');

```

```

p_strain = polyfit(log(h_list(1:end-1)), log(rms_strain_error_ref_list(1:end-1)), 1);
p_stress = polyfit(log(h_list(1:end-1)), log(rms_stress_error_ref_list(1:end-1)), 1);

```

```

fprintf('\n=====\\n');
fprintf('Estimated strain convergence order = %.12e\\n', p_strain(1));
fprintf('Estimated stress convergence order = %.12e\\n', p_stress(1));
fprintf('=====\\n');

```

```

function [node, elem] = generateMesh(L, a, nx, ny)

```



```

x = linspace(0, L, nx+1);
y = linspace(-a, a, ny+1);

node = zeros((nx+1)*(ny+1), 2);

count = 1;
for j = 1:ny+1
    for i = 1:nx+1
        node(count,:) = [x(i), y(j)];
        count = count + 1;
    end
end

elem = zeros(nx*ny, 4);

count = 1;
for j = 1:ny
    for i = 1:nx

        n1 = (j-1)*(nx+1) + i;
        n2 = n1 + 1;
        n3 = n2 + (nx+1);
        n4 = n1 + (nx+1);

        elem(count,:) = [n1 n2 n3 n4];
        count = count + 1;

    end
end

end

function Ke = Q4ElementStiffness(coord, D, thickness)

    gaussPoint = [-1/sqrt(3), 1/sqrt(3)];
    Ke = zeros(8,8);

```

```

for i = 1:2
    for j = 1:2

        xi = gaussPoint(i);
        eta = gaussPoint(j);

        dN_dxi = 1/4 * [-(1-eta), (1-eta), (1+eta), -(1+eta)];
        dN_deta = 1/4 * [-(1-xi), -(1+xi), (1+xi), (1-xi)];

        J = [dN_dxi; dN_deta] * coord;
        detJ = det(J);
        invJ = inv(J);

        dN = invJ * [dN_dxi; dN_deta];

        B = zeros(3,8);

        for k = 1:4
            B(1,2*k-1) = dN(1,k);
            B(2,2*k) = dN(2,k);
            B(3,2*k-1) = dN(2,k);
            B(3,2*k) = dN(1,k);
        end

        Ke = Ke + B' * D * B * thickness * detJ;

    end
end

end

function [x_strain, y_strain, eps_num, sig_num] =
computeElementStrainStressAtCenter(node, elem, U, D)

numElem = size(elem,1);

x_strain = zeros(numElem,1);

```

```

y_strain = zeros(numElem,1);
eps_num   = zeros(numElem,1);
sig_num    = zeros(numElem,1);

xi = 0;
eta = 0;

dN_dxi = 1/4 * [-(1-eta), (1-eta), (1+eta), -(1+eta)];
dN_deta = 1/4 * [-(1-xi), -(1+xi), (1+xi), (1-xi)];

for e = 1:numElem

    elemNode = elem(e,:);
    coord = node(elemNode,:);

    J = [dN_dxi; dN_deta] * coord;
    invJ = inv(J);

    dN = invJ * [dN_dxi; dN_deta];

    B = zeros(3,8);

    for k = 1:4
        B(1,2*k-1) = dN(1,k);
        B(2,2*k)    = dN(2,k);
        B(3,2*k-1) = dN(2,k);
        B(3,2*k)    = dN(1,k);
    end

    dof = zeros(8,1);
    for k = 1:4
        dof(2*k-1) = 2*elemNode(k)-1;
        dof(2*k)    = 2*elemNode(k);
    end

    Ue = U(dof);

    strain = B * Ue;

```

```
stress = D * strain;
```

```
x_strain(e) = mean(coord(:,1));
```

```
y_strain(e) = mean(coord(:,2));
```

```
eps_num(e)  = strain(1);
```

```
sig_num(e)  = stress(1);
```

```
end
```

```
end
```